How Reliable Is a Single Run of SHAP? Budget, Model and Instance Conditions on Tabular Data

Anonymous authors
Paper under double-blind review

Abstract

Shapley-value explanations of tabular models are usually produced by a single run of a randomised, budget-limited estimator and read as if they were deterministic. We ask when one run suffices, judged by what a reader uses: the set of top-k attributes. We study Kernel SHAP and permutation SHAP (`shap` 0.51.0) on 20 (dataset, model) cells (five public datasets by four models, $d = 8$ to 20), with 50 instances and 30 runs per instance each, against exact references (enumeration) where $d \leq 16$ and a high-budget reference otherwise. Error falls at about the Monte-Carlo rate, but top-3 sets need more budget than the error suggests: at the smallest budgets of our grid (20 to 60 coalitions) one Kernel SHAP run recovers the reference top-3 set in 0.43–0.87 of runs. At the library-default budget ($2d+2048$ coalitions), which is exact for $d \leq 11$ and a sampling estimate beyond, one run recovers it in 0.95–0.99 of runs on the 8 $d > 10$ cells (German Credit, ACS Employment), and the top-3 match probability is at least 0.9 for 0.88–0.98 of instances; this is evidence from two datasets, one trained model per cell. On those cells two models retrained on 80% subsamples agree on the top-3 set less often than two runs (0.09–0.91), under our retraining protocol. At matched compute, averaging four Kernel SHAP runs had a lower top-3 match probability than one run four times larger (pooled difference -0.044, 95% CI -0.063 to -0.028, 44 matched budget pairs); for permutation SHAP we found no difference (+0.001, -0.008 to +0.010). Subset checks on the library's default `l1_reg` and on the explained output (probability or log-odds) show that both change what is reproduced.

1 Introduction

Post-hoc feature attributions for tabular models are routinely computed with Kernel SHAP or permutation SHAP (Lundberg & Lee, 2017) and then read as *the* explanation of a prediction: the top few attributes are quoted in an audit, a credit decision or a model review. Both estimators are randomised and finite-budget, so two calls with different seeds return different vectors. The estimator literature characterises how the error of such estimates falls with the budget (Covert & Lee, 2020; Mitchell et al., 2021; Musco & Witter, 2024); what a practitioner has to decide is different: whether *the one run* they have made, at a budget they can afford, supports the statement they are about to make.

We study this decision on tabular data. The statement we evaluate is the one a reader acts on, the *top-k set* (and sign) of the attributes, not the vector error. We measure it against exact Shapley values where the game can be enumerated and against a validated high-budget reference otherwise, and we put it on the same scale as variation that practitioners already live with: retrained models, redrawn backgrounds and chance. Our contributions are:

- a measurement protocol for the reliability of a single run (population-level adequacy criterion with a stated Clopper–Pearson limitation, instance-cluster bootstrap, three comparison standards), applied to a grid of 20 (dataset, model) cells (Sections 3–4). What is new relative to the estimator literature is the decision-level target (top-k sets at the budgets a user actually chooses, including the library default) and its comparison with other sources of variation, not the error–budget relation, which we only confirm;

- the conditions under which one run suffices on these cells: budget, dimension (the library default budget is exact for $d \leq 11$ and a sampling estimate beyond), model, and the margin between the last attribute shown and the first one omitted;
- two subset checks on library choices (the default `l1_reg` on four cells, the explained output on 5) that show these choices change what is reproduced, reported as remarks;
- negative and null results: no gain from averaging repeats over one larger run, no confirmed advantage of a per-instance diagnostic over what exists, and the experiments we did not run (Section 5.1).

Everything in the paper is regenerated from run code (reproduction notes in the repository).

2 Related work

Estimating Shapley-value attributions. Kernel SHAP (Lundberg & Lee, 2017) estimates Shapley values by a weighted regression on sampled coalitions, and is exact once all coalitions are enumerated. Covert & Lee (2020) analyse its estimator, propose paired sampling as a variance reduction, and give convergence detection and uncertainty estimates for it; Slack et al. (2020) give a Bayesian variant of Kernel SHAP and LIME with credible intervals; Mitchell et al. (2021) study permutation sampling for Shapley values and Musco & Witter (2024) give a leverage-score variant with non-asymptotic guarantees. These works establish that the estimator is random and how its error can shrink with the budget; we take the error–budget relation as expected (Section 4) and ask a different question: whether *one* run at a budget a practitioner actually uses supports the explanation that is shown, judged by decision-level quantities (top-k sets, signs), against exact references and against other sources of variation. For tree ensembles the interventional value function admits fast exact computation (Lundberg et al., 2019); we use this only as a labelled comparison standard.

Instability under random sampling has been studied most for LIME: repeated explanations of the same instance disagree, and stability indices and deterministic variants have been proposed (Visani et al., 2022; Zafar & Khan, 2021). For Shapley-value estimators the corresponding evidence is the variance analysis above; what is new here is the top-k reliability of one run at the budgets a user selects, compared with the variation caused by retraining and by the background.

What is being estimated. The attribution depends on the value function, e.g. interventional versus observational conditional expectations (Chen et al., 2020), and on the background set (Yuan et al., 2022), who report fluctuation of SHAP values and rankings across random backgrounds for deep models on one clinical dataset. We fix the marginal (interventional) value function and a fixed background, so that estimator noise is separated from definitional choices, and treat background redraws as a separate source of variation.

Disagreement, robustness and multiplicity. Krishna et al. (2022) study disagreement between different explanation methods on the same model and define rank-, sign- and feature-agreement measures, which we reuse. Alvarez-Melis & Jaakkola (2018) study robustness of explanations to input perturbation, and Slack et al. (2019) show that LIME and SHAP can be fooled by classifiers that behave differently on the off-manifold samples these explainers query. Predictive multiplicity (Marx et al., 2020) documents that equally accurate models can disagree on individual predictions; we use retrained models as a yardstick for how much attribution variation a user already tolerates.

Stability measures. We compare top-k sets across runs with the chance-corrected stability of Nogueira et al. (2018) and rankings with rank-biased overlap (Webber et al., 2010).

3 Problem setting and measures

Practice under study. A deployer explains the prediction of a trained model f for an instance x by running a randomised Shapley-value estimator once, with a finite budget, and presents the resulting vector $\hat{\phi} \in \mathbb{R}^d$ as the reason for the prediction. We call one call of the explainer, with its own random seed, the same model, instance, background set and budget, a *run*.

Value function and reference. Throughout, $v(S;x) = \frac{1}{|\mathcal{B}|}\sum_{b\in\mathcal{B}} f(x_S, b_{\bar{S}})$ is the marginal (interventional) expectation of the predicted probability of the positive class over a fixed background set $\mathcal{B}$ of 50 training rows (Chen et al., 2020); features are the d original attributes (a one-hot group is masked jointly). The target of every run is the Shapley value ϕ^* of this game. Where $d \le 16$ we compute ϕ^* exactly by enumeration; for German Credit ($d = 20$) we use the mean of 10 high-budget Kernel SHAP runs with its standard error, checked against enumeration on a subset (Section 4). Because a deployer has no ϕ^*, we also report *run-vs-run* agreement, which needs no reference. The budget is counted in model rows actually evaluated by one explainer call, which is comparable across explainers; wall-clock time is reported separately.

Measures. Per instance we compute, over R runs: the relative ℓ_2 error $\|\hat{\phi} - \phi^*\|_2/\|\phi^*\|_2$; the probability $p_i^{(k)}$ that the top-k set (by $|\phi|$) of a run equals that of ϕ^* ("match") and the analogous probability for two distinct runs; top-k Jaccard; Kendall's τ; rank-biased overlap (RBO, $p = 0.9$) (Webber et al., 2010); the chance-corrected stability of the R top-k sets (Nogueira et al., 2018); and the fraction of the reference's top-k attributes whose sign a run flips. Metric names for agreement follow Krishna et al. (2022). The instance's *margin* is $(|\phi^*|_{(k)} - |\phi^*|_{(k+1)})/|\phi^*|_{(1)}$, the relative gap between the last attribute shown and the first one omitted.

Reliability criterion. A per-instance statement such as "the top-3 set is correct with probability ≥ 0.95" cannot be made from $R = 30$ runs: even 30/30 matches give a one-sided 95% Clopper–Pearson lower bound of only 0.905. Our headline quantity is therefore population-level: the mean of $\hat{p}_i^{(k)}$ over instances with an instance-cluster bootstrap CI, together with the fraction of instances with $\hat{p}_i^{(k)} \ge 0.8$ and ≥ 0.9, reported on the grid $k \in \{1, 3, 5\}$ rather than at one threshold. Headline cells and $k = 3$ were fixed in the plan in force when the main grid was launched, which is part of the delivery (Appendix C). Ties in $|\phi|$ are broken by attribute index; a single value $k = 3$ is used for every d, so cells are compared by what a user reading three attributes would see, not at a fixed fraction of d, and the chance level of a match (one over $\binom{d}{3}$: 0.018 at $d = 8$, below 0.001 at $d = 20$) is far below the values we discuss.

Comparison standards. A measure of run-to-run variation is judged against three standards, each computed with the same measure on the same scale: (i) *model-induced* variation, between models retrained on random 80% subsamples of the training set, explained with the exact value function where $d \le 16$ and, for German Credit ($d = 20$), with a single Kernel SHAP run of 10 000 coalitions, in the spirit of predictive multiplicity (Marx et al., 2020); (ii) *background-induced* variation under redrawn backgrounds of 50 and 200 rows; (iii) a *null* floor: attributions of random other instances and chance-level random k-subsets. Disagreement between different explanation methods (Krishna et al., 2022) was planned as a further standard (Kernel versus permutation versus tree-based attributions) but was not run; Kernel and permutation SHAP are compared only through their error and match curves against the same reference. In addition, a *compute-matched* comparison asks whether one run at budget mB beats the mean of m runs at budget B.

Statistics. The instance is the unit of inference; runs are nested in instances and are never bootstrapped alone. Cells (dataset × model) are never pooled silently. CIs are percentile intervals from an instance-cluster bootstrap (1000 resamples), and Wilson intervals for fractions of instances. Within a cell the model has one training seed, so statements are conditional on that model; training-seed variation appears only in the retrain standard.

4 Experiments

4.1 Setup

Data and models. We use public tabular binary-classification data (Appendix A, Table 2): Statlog German Credit ($d = 20$), the folktables (Ding et al., 2021) ACS Income ($d = 10$) and ACS Employment ($d = 16$) tasks (California, 2018, 1-year person file, 30 000-row subsample), Give Me Some Credit ($d = 10$, 20 000-row subsample, median imputation) and Pima Indians Diabetes ($d = 8$). Each dataset has a fixed stratified 70/30 split (seed 0); preprocessing is fit on the training part. Per

dataset we train an ℓ_2 logistic regression, a random forest, histogram gradient-boosted trees and a small MLP (64×64); test AUC is 0.71–0.90 across cells (Table 2). 50 random test instances are explained per cell.

Explainers and budgets. We use the `shap` 0.51.0 `KernelExplainer` (paired sampling, `l1_reg=False` in the main grid; the library's default `l1_reg` is examined separately on four cells) and the `PermutationExplainer`. Each run sets its own seed explicitly, since in this version `KernelExplainer`'s `seed` argument does not seed the sampler. The budget axis is the number of model rows evaluated by one call. Kernel SHAP is exact once the number of coalitions reaches $2^d - 2$, and the library's default of $2d + 2048$ coalitions is clipped to that, so for $d \leq 11$ (in our data, $d \leq 10$) the default budget is exact and has no run-to-run variance; we therefore study budgets *below* the default on these datasets, as a stress test (the smallest budgets, 20 to 60 coalitions, about 2.5 to 3 per attribute, are far below what a user would normally choose), and the default itself on the datasets with $d > 10$. Each (cell, explainer, budget, instance) has $R = 30$ independent runs.

Compute. A call takes between a few milliseconds and well under a second at the budgets used (Table 3). The study answers the budget question for cheap models only; we do not examine an expensive model or a background so large that a user can afford only a few hundred rows. The whole study took about 73 CPU-hours on the world's meter (per-step table in `REPRODUCE.md`).

4.2 Run-to-run error shrinks at the Monte-Carlo rate, but top-k sets need much more

Figure 1 shows the median relative ℓ_2 error and the probability that a single run's top-3 set equals the reference's, against the rows evaluated. Across the 20 cells, the log–log slope of the mean error against budget is -0.52 to -0.49 for permutation SHAP (the Monte-Carlo rate $-\frac{1}{2}$) and -0.97 to -0.64 for Kernel SHAP, which is steeper because its sampling is stratified and approaches enumeration; slopes are fitted only on budgets below half of the coalition space. Kernel SHAP at the smallest budgets occasionally returns a numerically singular solution (large errors), so we also report medians (Appendix A). Throughout, "budget" is the nominal number of coalitions (Kernel) or `max_evals` (permutation), and the two do not cost the same: a permutation call of nominal B evaluates a different number of rows than the $50B + 1$ of Kernel SHAP, so the two explainers should be compared at equal rows (Figure 1 is drawn against rows; the table of Appendix A lists both). Error is not what a user sees, however: at the smallest budgets the top-3 set of one run equals the exact top-3 set on average in only 0.43–0.87 of runs (Kernel) and 0.55–0.99 (permutation) depending on the cell, and in 0.95–1.00 and 0.89–1.00 at the largest budgets (Appendix Table 4). Top-1 is much easier (0.83–0.98 for Kernel at the smallest budget) than top-5 (0.27–0.78).

The library default where it is sampled. On the 8 cells with $d > 10$ (German Credit with four models; ACS Employment with four models) the default budget of $2d+2048$ coalitions (2080,2088) is itself an estimate, not an enumeration. A single default-budget run reproduces the exact-reference top-1 attribute in 0.97–1.00 of runs, the top-3 set in 0.95–0.99 and the top-5 set in 0.93–0.97; two default-budget runs give the same top-3 set in 0.94–0.99 of pairs and the chance-corrected stability of the top-3 sets (Nogueira et al., 2018) is 0.98–1.00. Signs are not the limiting factor: a run flips the sign of 0.000–0.007 of the reference's top-3 attributes at the smallest budgets and 0.00 at the default (Table 1).

4.3 Whether one run suffices depends on budget, model and the instance's margin

Population-level adequacy is far from uniform. The fraction of instances whose top-3 match probability is at least 0.9 ranges from 0.00 to 0.66 (Kernel) at the smallest budgets and from 0.86 to 1.00 at the largest, with the MLP and tree ensembles lagging the logistic regression in the same dataset (Appendix Table 4). The instance's own margin explains much of this (Figure 2): top-3 sets are far more reproducible for instances in the top tercile of the reference margin than in the bottom one, and the match probability is a nearly monotone function of the ratio z of the reference gap to the run-to-run standard deviation of that gap (pooled, Figure 2b; per cell with a normal-approximation reference curve, Appendix Figure 6). Because the standard deviation in z is estimated from the same runs

Data	Model	B	rows	top-1	top-3 [95% CI]	top-5	pair top-3	Nogueira	≥ 0.9 [Wilson]
German	gbm	2088	104401	1.00	0.99 [0.96, 1.00]	0.95	0.98	0.99	0.98 [0.90, 1.00]
German	lr	2088	104401	1.00	0.99 [0.97, 1.00]	0.97	0.98	0.99	0.96 [0.87, 0.99]
German	mlp	2088	104401	0.99	0.98 [0.96, 0.99]	0.96	0.96	0.99	0.92 [0.81, 0.97]
German	rf	2088	104401	1.00	0.95 [0.90, 0.98]	0.95	0.94	0.98	0.88 [0.76, 0.94]
ACS-Emp	gbm	2080	104001	1.00	0.99 [0.98, 1.00]	0.97	0.99	0.99	0.98 [0.90, 1.00]
ACS-Emp	lr	2080	104001	0.97	0.99 [0.98, 1.00]	0.95	0.99	1.00	0.98 [0.90, 1.00]
ACS-Emp	mlp	2080	104001	0.99	0.96 [0.92, 0.99]	0.93	0.95	0.98	0.92 [0.81, 0.97]
ACS-Emp	rf	2080	104001	0.99	0.97 [0.95, 0.99]	0.93	0.96	0.98	0.94 [0.84, 0.98]

Table 1: Kernel SHAP at the library-default budget $B = 2d + 2048$ coalitions (rows $= 50B + 1$ model evaluations) on the cells with $d > 10$: probability that one run's top-k set equals the exact-reference one, probability that two runs give the same top-3 set, and the fraction of instances with top-3 match probability ≥ 0.9 (Wilson 95% CI). Headline cells fixed in advance (Appendix C).

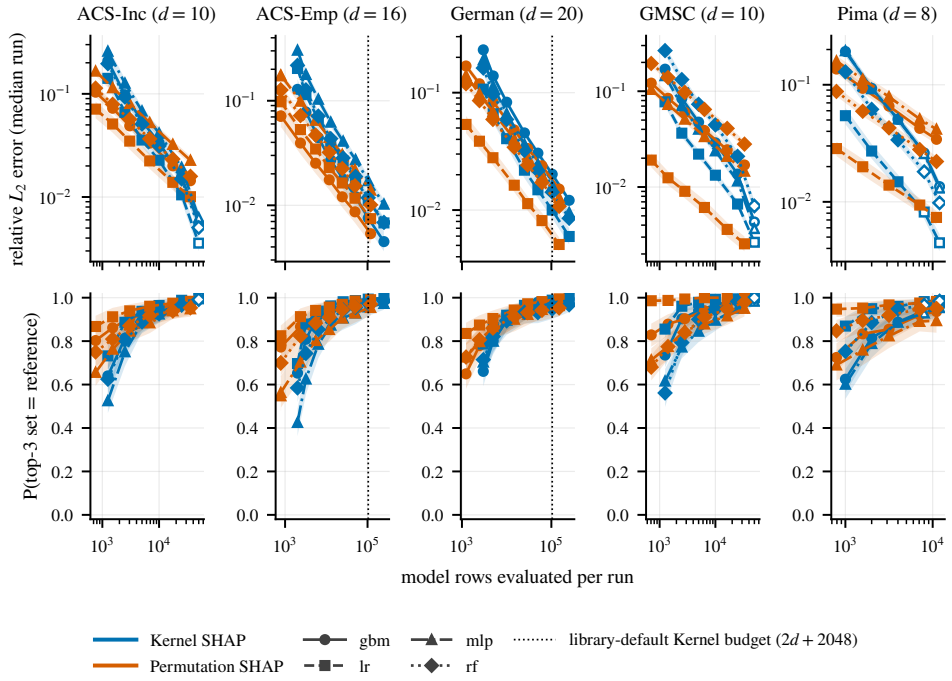


Figure 1: Median-run relative ℓ_2 error (top) and probability that a run's top-3 set equals the exact top-3 set (bottom) against model rows evaluated per run; colour: explainer (blue Kernel SHAP, orange permutation SHAP), line style and marker: model (see legend); dotted vertical line: library-default budget, drawn only where it is not exact ($d > 10$); shaded: 95% instance-cluster bootstrap CI; open markers lie outside the slope-fit range. 50 instances, $R = 30$ runs each, one trained model per curve.

whose match probability is plotted, this monotone relation is partly a consequence of the definitions and is not an independent test of a model; it shows that the margin relative to the run-to-run spread governs the match, for both estimators. A deployer can compute the single-run margin directly (gap between the kth and $(k{+}1)$th absolute attribution, relative to the largest); z needs repeated runs. At the default budget on the $d > 10$ Kernel SHAP cells, flagging instances whose single-run margin is below 0.02 flags 20% of the 400 instances and leaves 0.6% (95% CI up to 2.2%) of the unflagged ones with a top-3 match probability below 0.9, against 6.0% unreliable instances overall; at 0.05 it flags 42%. The base rate is low there, so this is a weak test: on the sub-default budgets of the $d \leq 10$ cells (base rate 18.5% unreliable) the same 0.02 threshold leaves 10.1% of unflagged Kernel instances unreliable (8.4–12.1%), and 16.7% for permutation SHAP. The thresholds were not fixed in advance. At the library-default budget on the $d > 10$ cells (Table 1), the fraction of instances

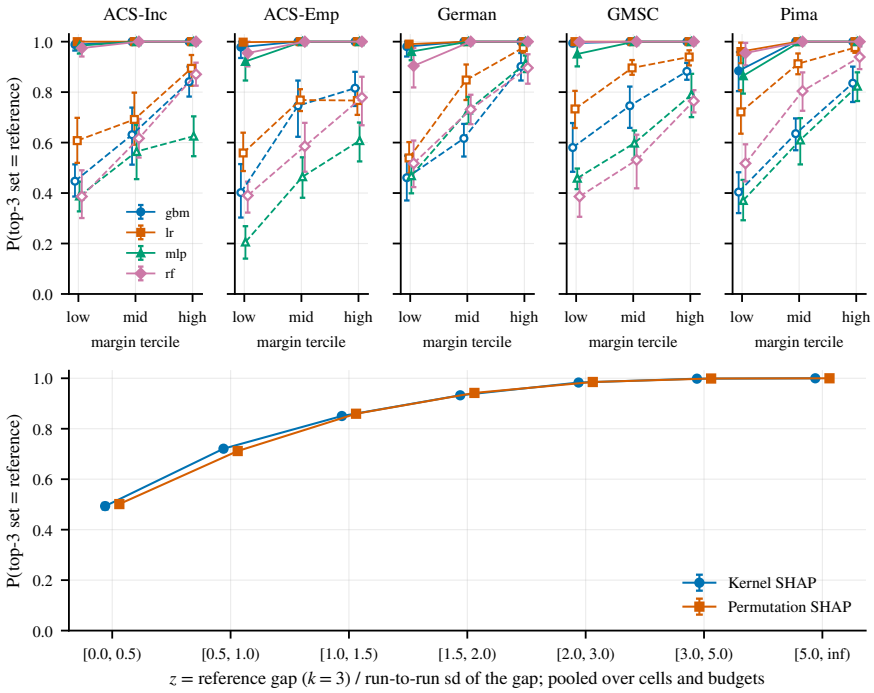


Figure 2: Top-3 match probability by tercile of the reference margin (a; Kernel SHAP, one line per model, dashed: smallest budget, solid: largest budget) and against z = reference gap/run-to-run sd of the gap, pooled over cells and budgets (b).

with top-3 match probability ≥ 0.9 is 0.88–0.98 across cells (Wilson 95% bounds 0.76–1.00): a single default-budget run is adequate for most instances, but for a minority of instances the top-3 set of one run is not reliably the reference's. Since $R = 30$ gives a Clopper–Pearson lower bound of only 0.905 even when all 30 runs agree, we state this criterion at the population level (the fraction of instances), not as a per-instance guarantee.

4.4 Comparison with other sources of variation

Figure 3 puts every source of variation on the same scale: the probability that two top-3 sets that *should* agree are identical. For two runs at the largest budget this is close to one. The background standard was run in full for $d \leq 10$ (ACS Income, Give Me Some Credit, Pima); for German Credit and the ACS Employment logistic regression only a reduced version exists (three redraws of 50 rows, 30 instances, top-3 agreement 0.51–0.70), so on the sampled-regime cells the comparison rests mainly on the retrain standard. Two models retrained on random 80% subsamples of the training data (explained with the exact value function where $d \leq 16$, and by one Kernel SHAP run of 10 000 coalitions for German Credit, where the estimator's variance is an estimated 0.036% at most of the disagreement) agree on 0.09–0.96 over cells, two background redraws of 50 rows on 0.31–0.79 and of 200 rows on 0.61–0.86, whereas attributions of random other instances agree on 0.01–0.26 and random 3-subsets on 0.00–0.02 (chance). Thus, under this retraining protocol (80% subsamples, five seeds, one data split) and these background draws, at the larger budgets explainer noise is smaller than the variation they induce; at the smallest budgets it is of the same order or larger. This is strongest in the sampled regime: on the $d > 10$ cells the retrained models agree on the top-3 set in 0.09–0.91 of instances, below the 0.94–0.99 of two default-budget runs in 8 of 8 cells. The result depends on the subsample fraction: on the $d > 10$ cells, agreement at 50% subsamples is 0.05–0.89 (7 such cells) and at 95% 0.18–0.98 (7 cells: the four German Credit models and the ACS Employment logistic regression, forest and MLP), below two runs in all but the ACS Employment logistic regression (Appendix Table 6). On those datasets the variation that matters to a user is the one that comes with the model fit, not with the Kernel SHAP seed.

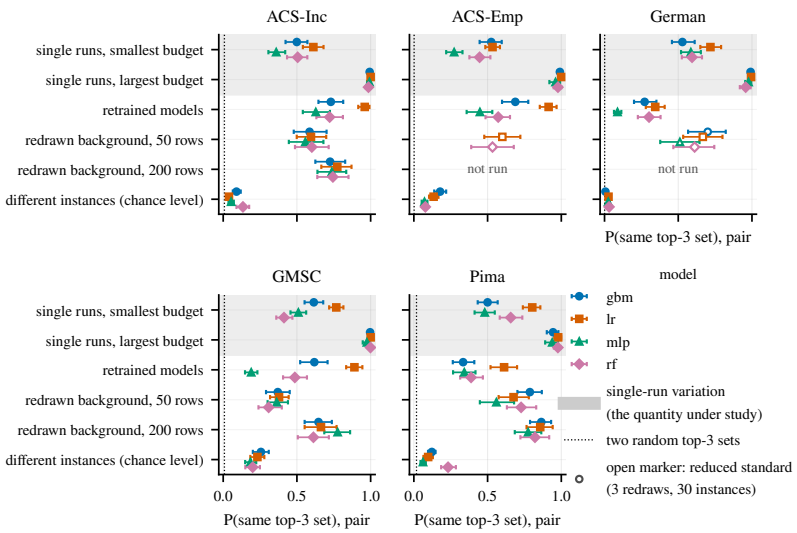


Figure 3: Probability that two top-3 sets are identical, for pairs that should agree: two explainer runs (smallest, largest budget), two retrained models, two backgrounds (50, 200 rows), and the null of two random instances and chance for random 3-subsets (dotted). One point per model, 95% CI. Open markers: reduced background standard (3 redraws, 30 instances; German Credit and ACS-Emp logistic regression); 200-row redraws were run only for $d \leq 10$ ("not run" elsewhere).

4.5 Library defaults and the explained output

Two choices a library user can make without noticing changed the result by more than the seed did in the cells where we tested them; these are remarks on a subset of cells (`l1_reg` on four, the output on 5), not part of the reliability result. (i) With the default `l1_reg` (a LARS pre-selection of at most ten attributes), Kernel SHAP on German Credit (logistic regression, gradient boosting, MLP) does not converge to the Shapley values: at the default budget the median relative error is 0.17–0.20 against 0.01–0.02 with `l1_reg=False` and does not shrink with the budget (0.17–0.20 at 5000 coalitions); the top-3 set differs from that of the same-seed `l1_reg=False` run in 0.07–0.12 of runs at the default budget (Appendix B, Figure 7), a bias of the sparsified target rather than noise. On the ACS Employment logistic regression the effect is small (median error 0.07 against 0.007 at 5000 coalitions; top-3 differs in 0.004 of runs at the default). (ii) Attributions of $P(y{=}1)$ and of the log-odds are different games: on the 5 cells where we ran both, the exact top-3 sets coincide for only 0.50–0.86 of instances, while run-to-run reliability is similar or higher on log-odds (Table 9; change in top-3 match probability, log-odds minus probability, averaged over budgets: -0.01 to 0.10 per cell and estimator). (iii) We checked the German Credit reference against enumeration (2^{20} coalitions) on 26 instances of 2 models (logistic regression and gradient boosting): the 20 000-coalition reference is within 0.2% of the exact vector in relative ℓ_2 distance and has the exact top-3 and top-5 sets in 26 of 26 checks (Table 8).

4.6 Repeating versus enlarging

At matched compute, is the mean of c runs at budget b better than one run at $\approx cb$? The budgets of our grid are not generally a factor of c apart, so we keep only pairs whose compute ratio lies within $[0.9, 1.1]$ (for $c = 4$, 68 of 216 pairs; the remainder are not compute-matched and are not used). We pool the per-pair differences in top-3 match probability (repeat minus enlarge) with a bootstrap that resamples cells and then instances within cells; a second interval resamples the five datasets first. For Kernel SHAP the pooled difference is -0.027 at $c = 2$ (95% CI -0.035 to -0.020; datasets-first -0.038 to -0.015; 76 pairs), -0.044 at $c = 4$ (-0.063 to -0.028; -0.071 to -0.017; 44 pairs) and -0.084 at $c = 8$ (-0.107 to -0.061; -0.117 to -0.047; 28 pairs): the single larger run is better, and the repeat wins in no cell at $c = 2, 8$ and in 3 of 20 cells at $c = 4$. For permutation SHAP there is no difference we can resolve: -0.001 at $c = 2$ (-0.004 to +0.003; 60 pairs) and +0.001 at $c = 4$ (-0.008 to +0.010;

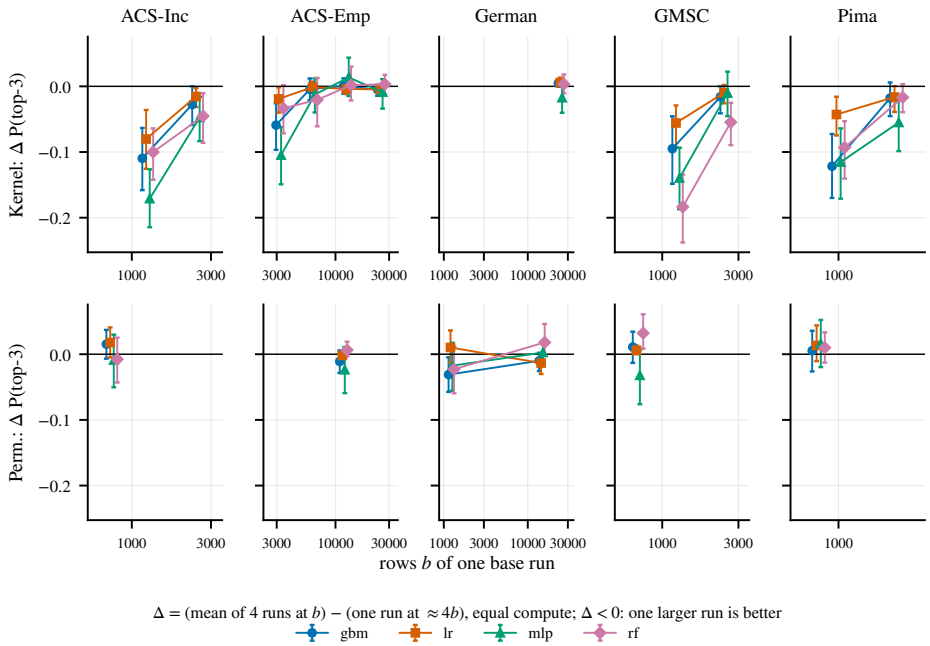


Figure 4: Top-3 match probability: difference between the mean of 4 runs at budget b and one run at $\approx 4b$ rows (matched pairs only, ratio 0.9–1.1), 95% CI.

24 pairs), and there are no matched pairs at $c = 8$ (Figure 4). The conclusion is limited to matched pairs, these two estimators and $c = 2, 4, 8$.

4.7 Predicting unreliable instances (exploratory)

We evaluated whether per-instance quantities available at deployment flag instances whose top-k set is unreliable: the instance's own margin computed from a single run, and the signal-to-noise z and match probability estimated from repeats. Median AUCs over cells and settings are 0.94 (own margin), 0.97 (repeat z) and 0.97 (repeat match probability) (Appendix, Figure 5); this analysis was marked exploratory in advance and has not been compared with the variance estimate of Covert & Lee (2020), so we make no claim that it improves on existing uncertainty estimates.

5 Discussion and limitations

What the evidence supports. On German Credit (four models) and ACS Employment (gbm, lr, mlp, rf), a single Kernel SHAP run at the library default is adequate for most instances and its noise is smaller than the change that a retrained model brings under our retraining protocol; this is two-dataset evidence on one trained model per cell; the cases to worry about are instances with a small margin between the last attribute shown and the first omitted, and any setting below the default budget. For $d \leq 11$ the default is exact; in our stress-test grid below the default, top-k stability degrades while the error still looks small, but this depends on where our grid starts. These are statements about the cells we ran (Table 2); they are not a guarantee for other datasets, models or d.

What it means for practice. If an explanation is quoted as a top-k set, report the margin or repeat a small number of runs on the instance; for Kernel SHAP, averaging m runs was not a better use of compute than one run m times larger, and for permutation SHAP the two were indistinguishable (Section 4). Check the library defaults: in the cells where we tested them (a subset), `l1_reg` and the explained output changed the result by more than the seed did.

5.1 LIMITATIONS

- **Scope of inference.** One trained model per cell and 50 explained instances per cell: statements are conditional on that model and apply to instances of it. Five datasets (ACS: California 2018 only; two are tiny), four model classes; no deep or very high-dimensional data.
- **Regime.** For $d \leq 10$ we study sub-default budgets because the library default is exact; the default-budget claims rest on the cells with $d > 10$ only.
- **Repeat versus enlarge.** The comparison exists only at the budget pairs of our grid whose compute ratio is within $[0.9, 1.1]$ for $c = 2, 4, 8$ repeats (Section 4), has no matched permutation pairs at $c = 8$, and its intervals rest on five datasets; we make no statement for other ratios.
- **Dependence between cells.** Seeds are the same function of (instance, budget index, run) in every cell, so the cells of one dataset sample the same coalitions for the same instances; the rank correlation of run noise between two models of one dataset is on average 0.27 (Kernel) and 0.13 (permutation), at most 0.73, against about zero with shuffled run index. The effective number of independent units is closer to the five datasets than to the 20 cells.
- **Retrain standard.** One protocol (80% subsamples, five seeds, one split); the subsample fraction was varied (50%, 95%) only for 11 cells (appendix), and the retrained models share most of their data. The background standard is reduced (three redraws) for $d > 10$.
- **Reference.** For German Credit the reference is itself an estimate (the mean of 10 runs at 20 000 coalitions); it was checked against enumeration on 26 instances of two models, which supports "no gross bias" and nothing more.
- **Library-specific.** All results are for `shap` 0.51.0 with its paired sampling and its seeding behaviour; the default `l1_reg` and log-odds checks cover a subset of cells (Section 4).
- **Not done.** The leverage-score estimator of Musco & Witter (2024), a tree-based exact comparison, the Covert–Lee variance diagnostic (Covert & Lee, 2020), the Arrhythmia stress test and German Credit budgets beyond 5000 were not run, and the per-instance diagnostic of Section 4.7 is exploratory.

REPRODUCIBILITY STATEMENT

Code, run logs, result tables and scripts for every table and figure are in the delivered repository; `REPRODUCE.md` gives the environment (`shap` 0.51.0), data sources and checksums, seeds, commands, run times, failed or unfinished runs and a table/figure-to-script map. Compute exceeded the plan's nominal 24 CPU-hours (about 73 CPU-hours on the world's meter, mostly the grids on the $d \geq 16$ cells); the per-step table is in `REPRODUCE.md`. Numbers in the text are written into `paper/numbers.tex` by `paper/make_numbers.py` from `experiment/results/tables`. The report role did not search for, access or use the withheld published paper on this question or anything citing it.

ETHICS STATEMENT

The study uses public, de-identified tabular datasets (UCI German Credit and Pima, Give Me Some Credit, US Census folktables ACS for California 2018) under their stated terms; no new data on people were collected. The models are trained for measurement only, not for decisions about people. Reading single-run explanations as exact may mislead; we report where one run suffices so that the result is not used to dismiss adequate explanations either.

AI USE STATEMENT

In this work, we used generative AI tools for proposing and refining the research questions, designing the methodology and experiments, implementing the methods and analysis code, running the experiments, interpreting the results and analysing the data. The work was carried out by AI agents

(Claude models) acting as separate roles of an automated research organisation (context and literature, study plan, feasibility review, analysis, figures and tables, venue formatting, report writing, and internal and external review); the research objective was given to the organisation, and no human author took part in the work itself. We did not use generative AI tools to collect new data; the datasets are public and were downloaded as described in the reproduction notes. Additionally, we used generative AI tools for writing and editing code, drafting and revising the text, searching and reading the literature, formatting references, and creating figures and tables. We have reviewed all AI-assisted work through the checks built into the process: every number in the text is written from the result tables by a script, the experiment pipeline regenerates the tables from raw run files, cited references were looked up in public bibliographic sources, and a role separate from the authoring roles reviewed each delivery against the objective. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REFERENCES

David Alvarez-Melis and Tommi S. Jaakkola. On the robustness of interpretability methods, 2018.

Hugh Chen, Joseph D. Janizek, Scott Lundberg, and Su-In Lee. True to the model or true to the data?, 2020.

Ian Covert and Su-In Lee. Improving KernelSHAP: Practical Shapley value estimation via linear regression, 2020.

Frances Ding, Moritz Hardt, John Miller, and Ludwig Schmidt. Retiring adult: New datasets for fair machine learning. In *Advances in Neural Information Processing Systems 34*, 2021.

Satyapriya Krishna, Tessa Han, Alex Gu, Steven Wu, Shahin Jabbari, and Himabindu Lakkaraju. The disagreement problem in explainable machine learning: A practitioner's perspective, 2022.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, 2017.

Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, et al. Explainable AI for trees: From local explanations to global understanding, 2019.

Charles T. Marx, Flavio du Pin Calmon, and Berk Ustun. Predictive multiplicity in classification. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

Rory Mitchell, Joshua Cooper, Eibe Frank, and Geoffrey Holmes. Sampling permutations for Shapley value estimation, 2021.

Christopher Musco and R. Teal Witter. Provably accurate Shapley value estimation via leverage score sampling, 2024.

Sarah Nogueira, Konstantinos Sechidis, and Gavin Brown. On the stability of feature selection algorithms. *Journal of Machine Learning Research*, 18(174), 2018.

Dylan Slack, Sophie Hilgard, Emily Jia, Sameer Singh, and Himabindu Lakkaraju. Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods, 2019.

Dylan Slack, Sophie Hilgard, Sameer Singh, and Himabindu Lakkaraju. Reliable post hoc explanations: Modeling uncertainty in explainability, 2020.

Giorgio Visani, Enrico Bagli, Federico Chesani, Alessandro Poluzzi, and Davide Capuzzo. Statistical stability indices for LIME: Obtaining reliable explanations for machine learning models. *Journal of the Operational Research Society*, 73(1):91–101, 2022.

William Webber, Alistair Moffat, and Justin Zobel. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems*, 28(4), 2010.

Han Yuan, Mingxuan Liu, Lican Kang, Chenkui Miao, and Ying Wu. An empirical study of the effect of background data size on the stability of SHapley Additive exPlanations (SHAP) for deep learning models, 2022.

Muhammad Rehman Zafar and Naimul Khan. Deterministic local interpretable model-agnostic explanations for stable explainability. *Machine Learning and Knowledge Extraction*, 3(3):525–541, 2021.

A SETUP, COST AND FULL RESULTS

Data	Model	d	Inst.	Runs R	Backgr. B	Test AUC	Test acc.	Reference
ACS-Inc	gbm	10	50	30	50	0.897	0.823	enumeration
ACS-Inc	lr	10	50	30	50	0.869	0.791	enumeration
ACS-Inc	mlp	10	50	30	50	0.879	0.799	enumeration
ACS-Inc	rf	10	50	30	50	0.891	0.816	enumeration
ACS-Emp	gbm	16	50	30	50	0.900	0.819	enumeration
ACS-Emp	lr	16	50	30	50	0.867	0.796	enumeration
ACS-Emp	mlp	16	50	30	50	0.891	0.810	enumeration
ACS-Emp	rf	16	50	30	50	0.897	0.817	enumeration
German	gbm	20	50	30	50	0.783	0.740	Kernel 20000×10 (rel. SE 0.002)
German	lr	20	50	30	50	0.800	0.760	Kernel 20000×10 (rel. SE 0.001)
German	mlp	20	50	30	50	0.756	0.730	Kernel 20000×10 (rel. SE 0.001)
German	rf	20	50	30	50	0.798	0.747	Kernel 20000×10 (rel. SE 0.001)
GMSC	gbm	10	50	30	50	0.873	0.935	enumeration
GMSC	lr	10	50	30	50	0.711	0.932	enumeration
GMSC	mlp	10	50	30	50	0.844	0.933	enumeration
GMSC	rf	10	50	30	50	0.860	0.934	enumeration
Pima	gbm	8	50	30	50	0.828	0.797	enumeration
Pima	lr	8	50	30	50	0.871	0.775	enumeration
Pima	mlp	8	50	30	50	0.746	0.710	enumeration
Pima	rf	8	50	30	50	0.847	0.775	enumeration

Table 2: Cells: dimension d, instances explained, runs R per instance, background size, test AUC and accuracy of the main model, reference type.

		Kernel SHAP				Permutation SHAP			
Data	Model	$B_{\min}$ (rows)	s/call	$B_{\max}$ (rows)	s/call	$B_{\min}$ (rows)	s/call	$B_{\max}$ (rows)	s/call
Pima	lr	20 (1001)	0.004	240 (12001)	0.009	20 (790)	0.001	240 (11059)	0.013
Pima	rf	20 (1001)	0.018	240 (12001)	0.039	20 (790)	0.008	240 (11059)	0.092
Pima	gbm	20 (1001)	0.015	240 (12001)	0.097	20 (790)	0.009	240 (11059)	0.112
Pima	mlp	20 (1001)	0.005	240 (12001)	0.028	20 (790)	0.002	240 (11059)	0.021
GMSC	lr	25 (1251)	0.005	1000 (50001)	0.031	25 (707)	0.002	1000 (33231)	0.044
GMSC	rf	25 (1251)	0.023	1000 (50001)	0.178	25 (707)	0.009	1000 (33231)	0.420
GMSC	gbm	25 (1251)	0.009	1000 (50001)	0.147	25 (707)	0.004	1000 (33231)	0.154
GMSC	mlp	25 (1251)	0.005	1000 (50001)	0.130	25 (707)	0.002	1000 (33231)	0.066
ACS-Inc	lr	25 (1251)	0.005	1000 (50001)	0.061	25 (760)	0.002	1000 (35709)	0.057
ACS-Inc	rf	25 (1251)	0.026	1000 (50001)	0.248	25 (760)	0.011	1000 (35709)	0.495
ACS-Inc	gbm	25 (1251)	0.018	1000 (50001)	0.705	25 (760)	0.009	1000 (35709)	0.392
ACS-Inc	mlp	25 (1251)	0.006	1000 (50001)	0.107	25 (760)	0.003	1000 (35709)	0.086
German	lr	60 (3001)	0.009	5000 (250001)	0.591	60 (1232)	0.003	5000 (149111)	0.227
German	rf	60 (3001)	0.030	5000 (250001)	1.036	60 (1232)	0.011	5000 (149111)	1.152
German	gbm	60 (3001)	0.037	5000 (250001)	4.599	60 (1232)	0.015	5000 (149111)	1.698
German	mlp	60 (3001)	0.012	5000 (250001)	1.072	60 (1232)	0.004	5000 (149111)	0.349
ACS-Emp	lr	40 (2001)	0.008	5000 (250001)	0.776	40 (787)	0.002	5000 (118813)	0.249
ACS-Emp	rf	40 (2001)	0.030	5000 (250001)	1.431	40 (787)	0.011	5000 (118813)	1.590
ACS-Emp	gbm	40 (2001)	0.025	5000 (250001)	5.534	40 (787)	0.010	5000 (118813)	1.337
ACS-Emp	mlp	40 (2001)	0.010	5000 (250001)	1.299	40 (787)	0.003	5000 (118813)	0.375

Table 3: Single-thread seconds per explainer call at the smallest and largest budget B with the rows evaluated in parentheses. B is the nominal budget (Kernel: coalitions; permutation: `max_evals`), and the number of model rows actually evaluated differs between the explainers at the same B; timed on a shared, oversubscribed host, so indicative.

A.1 SMALLEST AND LARGEST BUDGET PER CELL

A.2 FULL TOP-k MATCH GRID

			smallest budget				largest budget			
Data	Model	Expl.	B	Rows	$\bar{p}$ [95% CI]	$\geq.9$	B	Rows	$\bar{p}$ [95% CI]	$\geq.9$
ACS-Inc	gbm	Kernel	25	1251	0.64 [0.58, 0.70]	0.18	1000	50001	1.00 [0.99, 1.00]	0.98
ACS-Inc	gbm	Perm.	25	760	0.80 [0.74, 0.86]	0.50	1000	35709	0.98 [0.97, 1.00]	0.94
ACS-Inc	lr	Kernel	25	1251	0.73 [0.67, 0.79]	0.30	1000	50001	1.00 [1.00, 1.00]	1.00
ACS-Inc	lr	Perm.	25	760	0.87 [0.82, 0.92]	0.60	1000	35709	0.98 [0.96, 1.00]	0.96
ACS-Inc	mlp	Kernel	25	1251	0.53 [0.47, 0.59]	0.06	1000	50001	0.99 [0.98, 1.00]	0.98
ACS-Inc	mlp	Perm.	25	760	0.66 [0.59, 0.72]	0.22	1000	35709	0.95 [0.91, 0.98]	0.84
ACS-Inc	rf	Kernel	25	1251	0.62 [0.55, 0.69]	0.16	1000	50001	0.99 [0.98, 1.00]	0.98
ACS-Inc	rf	Perm.	25	760	0.75 [0.68, 0.82]	0.40	1000	35709	0.95 [0.92, 0.98]	0.82
ACS-Emp	gbm	Kernel	40	2001	0.65 [0.58, 0.73]	0.20	5000	250001	0.99 [0.98, 1.00]	0.98
ACS-Emp	gbm	Perm.	40	787	0.78 [0.71, 0.85]	0.56	5000	118813	0.99 [0.97, 1.00]	0.98
ACS-Emp	lr	Kernel	40	2001	0.70 [0.65, 0.74]	0.10	5000	250001	1.00 [1.00, 1.00]	1.00
ACS-Emp	lr	Perm.	40	787	0.80 [0.74, 0.86]	0.54	5000	118813	0.99 [0.98, 1.00]	0.96
ACS-Emp	mlp	Kernel	40	2001	0.43 [0.36, 0.49]	0.02	5000	250001	0.97 [0.94, 1.00]	0.92
ACS-Emp	mlp	Perm.	40	787	0.55 [0.48, 0.62]	0.14	5000	118813	0.95 [0.92, 0.99]	0.90
ACS-Emp	rf	Kernel	40	2001	0.58 [0.52, 0.65]	0.12	5000	250001	0.98 [0.96, 1.00]	0.94
ACS-Emp	rf	Perm.	40	787	0.70 [0.63, 0.76]	0.32	5000	118813	0.97 [0.94, 0.99]	0.90
German	gbm	Kernel	60	3001	0.66 [0.59, 0.73]	0.20	5000	250001	0.99 [0.98, 1.00]	0.98
German	gbm	Perm.	60	1232	0.65 [0.58, 0.73]	0.28	5000	149111	0.98 [0.96, 1.00]	0.98
German	lr	Kernel	60	3001	0.79 [0.72, 0.84]	0.50	5000	250001	1.00 [0.99, 1.00]	0.98
German	lr	Perm.	60	1232	0.84 [0.78, 0.88]	0.52	5000	149111	0.99 [0.99, 1.00]	1.00
German	mlp	Kernel	60	3001	0.70 [0.64, 0.76]	0.26	5000	250001	0.99 [0.97, 1.00]	0.92
German	mlp	Perm.	60	1232	0.73 [0.65, 0.79]	0.34	5000	149111	0.96 [0.93, 0.99]	0.88
German	rf	Kernel	60	3001	0.71 [0.65, 0.77]	0.30	5000	250001	0.97 [0.93, 0.99]	0.90
German	rf	Perm.	60	1232	0.73 [0.66, 0.79]	0.34	5000	149111	0.95 [0.91, 0.99]	0.90
GMSC	gbm	Kernel	25	1251	0.74 [0.68, 0.79]	0.34	1000	50001	1.00 [0.99, 1.00]	1.00
GMSC	gbm	Perm.	25	707	0.83 [0.77, 0.88]	0.54	1000	33231	0.98 [0.96, 1.00]	0.94
GMSC	lr	Kernel	25	1251	0.86 [0.82, 0.89]	0.54	1000	50001	1.00 [1.00, 1.00]	1.00
GMSC	lr	Perm.	25	707	0.99 [0.97, 1.00]	0.94	1000	33231	1.00 [1.00, 1.00]	1.00
GMSC	mlp	Kernel	25	1251	0.62 [0.56, 0.67]	0.12	1000	50001	0.98 [0.96, 1.00]	0.96
GMSC	mlp	Perm.	25	707	0.71 [0.65, 0.77]	0.28	1000	33231	0.95 [0.91, 0.98]	0.88
GMSC	rf	Kernel	25	1251	0.56 [0.50, 0.62]	0.00	1000	50001	1.00 [1.00, 1.00]	1.00
GMSC	rf	Perm.	25	707	0.68 [0.62, 0.73]	0.18	1000	33231	0.98 [0.95, 1.00]	0.96
Pima	gbm	Kernel	20	1001	0.62 [0.56, 0.69]	0.18	240	12001	0.96 [0.93, 0.99]	0.88
Pima	gbm	Perm.	20	790	0.72 [0.66, 0.79]	0.34	240	11059	0.94 [0.90, 0.97]	0.82
Pima	lr	Kernel	20	1001	0.87 [0.82, 0.91]	0.66	240	12001	0.99 [0.97, 1.00]	0.96
Pima	lr	Perm.	20	790	0.95 [0.91, 0.98]	0.84	240	11059	0.98 [0.96, 1.00]	0.92
Pima	mlp	Kernel	20	1001	0.60 [0.53, 0.67]	0.16	240	12001	0.95 [0.92, 0.98]	0.86
Pima	mlp	Perm.	20	790	0.69 [0.62, 0.76]	0.30	240	11059	0.89 [0.84, 0.94]	0.72
Pima	rf	Kernel	20	1001	0.75 [0.69, 0.81]	0.36	240	12001	0.98 [0.97, 1.00]	0.96
Pima	rf	Perm.	20	790	0.85 [0.80, 0.90]	0.58	240	11059	0.95 [0.92, 0.98]	0.86

Table 4: Mean over instances of the probability $\bar{p}$ that one run's top-3 set equals the exact top-3 set, with 95% instance-cluster bootstrap CI, and the fraction of instances for which this probability is ≥ 0.9, at the smallest and largest nominal budget B of each cell (Kernel: coalitions; permutation: `max_evals`); the Rows column gives the mean number of model rows evaluated per run, which differs between the explainers at the same B, so compare them at equal rows. Full grid: Appendix A.2.

					top-1		top-3		top-5	
Data	Model	Expl.	B	Rows	$\bar{p}$ [95% CI]	$\geq.9$	$\bar{p}$ [95% CI]	$\geq.9$	$\bar{p}$ [95% CI]	$\geq.9$
ACS-Inc	gbm	Kernel	25	1251	0.89 [0.85, 0.93]	0.70	0.64 [0.58, 0.70]	0.18	0.39 [0.34, 0.44]	0.02
ACS-Inc	gbm	Kernel	50	2501	0.96 [0.92, 0.99]	0.92	0.81 [0.76, 0.87]	0.50	0.68 [0.62, 0.74]	0.18
ACS-Inc	gbm	Kernel	100	5001	0.98 [0.94, 1.00]	0.94	0.90 [0.85, 0.94]	0.70	0.82 [0.76, 0.87]	0.50
ACS-Inc	gbm	Kernel	200	10001	0.98 [0.95, 1.00]	0.96	0.93 [0.89, 0.97]	0.80	0.91 [0.87, 0.95]	0.72
ACS-Inc	gbm	Kernel	500	25001	0.99 [0.98, 1.00]	0.98	0.97 [0.94, 0.99]	0.92	0.97 [0.94, 0.99]	0.88
ACS-Inc	gbm	Kernel	1000	50001	1.00 [0.99, 1.00]	0.98	1.00 [0.99, 1.00]	0.98	0.99 [0.97, 1.00]	0.98
ACS-Inc	gbm	Perm.	25	760	0.94 [0.90, 0.97]	0.80	0.80 [0.74, 0.86]	0.50	0.71 [0.64, 0.78]	0.34
ACS-Inc	gbm	Perm.	50	1520	0.95 [0.92, 0.98]	0.90	0.86 [0.81, 0.91]	0.62	0.80 [0.75, 0.85]	0.46
ACS-Inc	gbm	Perm.	100	3039	0.97 [0.93, 0.99]	0.92	0.90 [0.87, 0.94]	0.72	0.87 [0.83, 0.91]	0.64
ACS-Inc	gbm	Perm.	200	6838	0.98 [0.95, 1.00]	0.94	0.93 [0.89, 0.97]	0.78	0.93 [0.89, 0.96]	0.76
ACS-Inc	gbm	Perm.	500	17474	0.98 [0.95, 1.00]	0.94	0.97 [0.94, 0.99]	0.92	0.96 [0.93, 0.98]	0.86
ACS-Inc	gbm	Perm.	1000	35709	0.98 [0.96, 1.00]	0.94	0.98 [0.97, 1.00]	0.94	0.97 [0.95, 0.99]	0.96
ACS-Inc	lr	Kernel	25	1251	0.89 [0.85, 0.93]	0.70	0.73 [0.67, 0.79]	0.30	0.61 [0.54, 0.67]	0.14
ACS-Inc	lr	Kernel	50	2501	0.96 [0.93, 0.99]	0.92	0.89 [0.84, 0.93]	0.68	0.83 [0.77, 0.88]	0.48
ACS-Inc	lr	Kernel	100	5001	0.98 [0.96, 1.00]	0.94	0.94 [0.89, 0.97]	0.82	0.92 [0.87, 0.95]	0.78
ACS-Inc	lr	Kernel	200	10001	0.98 [0.96, 1.00]	0.94	0.96 [0.93, 0.99]	0.92	0.95 [0.92, 0.98]	0.86
ACS-Inc	lr	Kernel	500	25001	1.00 [0.99, 1.00]	0.98	0.99 [0.97, 1.00]	0.96	0.99 [0.97, 1.00]	0.94
ACS-Inc	lr	Kernel	1000	50001	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00
ACS-Inc	lr	Perm.	25	760	0.91 [0.86, 0.95]	0.72	0.87 [0.82, 0.92]	0.60	0.86 [0.80, 0.91]	0.62

Data	Model	Expl.	B	Rows	top-1 $\bar{p}$ [95% CI]	top-1 $\geq.9$	top-3 $\bar{p}$ [95% CI]	top-3 $\geq.9$	top-5 $\bar{p}$ [95% CI]	top-5 $\geq.9$
ACS-Inc	lr	Perm.	50	1520	0.95 [0.92, 0.98]	0.86	0.91 [0.87, 0.95]	0.80	0.92 [0.87, 0.95]	0.72
ACS-Inc	lr	Perm.	100	3039	0.97 [0.95, 0.99]	0.88	0.94 [0.90, 0.97]	0.80	0.95 [0.91, 0.97]	0.86
ACS-Inc	lr	Perm.	200	6838	0.98 [0.96, 1.00]	0.96	0.96 [0.93, 0.99]	0.88	0.95 [0.92, 0.98]	0.86
ACS-Inc	lr	Perm.	500	17474	0.99 [0.99, 1.00]	0.98	0.97 [0.95, 1.00]	0.94	0.98 [0.96, 1.00]	0.92
ACS-Inc	lr	Perm.	1000	35709	1.00 [0.99, 1.00]	0.98	0.98 [0.96, 1.00]	0.96	0.99 [0.97, 1.00]	0.96
ACS-Inc	mlp	Kernel	25	1251	0.88 [0.83, 0.92]	0.72	0.53 [0.47, 0.59]	0.06	0.30 [0.26, 0.36]	0.00
ACS-Inc	mlp	Kernel	50	2501	0.96 [0.93, 0.98]	0.84	0.75 [0.68, 0.81]	0.38	0.54 [0.48, 0.60]	0.14
ACS-Inc	mlp	Kernel	100	5001	0.98 [0.97, 0.99]	0.92	0.89 [0.84, 0.93]	0.68	0.72 [0.66, 0.78]	0.32
ACS-Inc	mlp	Kernel	200	10001	0.99 [0.98, 1.00]	0.98	0.92 [0.89, 0.96]	0.74	0.85 [0.81, 0.89]	0.54
ACS-Inc	mlp	Kernel	500	25001	1.00 [1.00, 1.00]	1.00	0.98 [0.96, 0.99]	0.94	0.93 [0.90, 0.96]	0.82
ACS-Inc	mlp	Kernel	1000	50001	1.00 [1.00, 1.00]	1.00	0.99 [0.98, 1.00]	0.98	0.98 [0.95, 1.00]	0.94
ACS-Inc	mlp	Perm.	25	760	0.94 [0.91, 0.97]	0.82	0.66 [0.59, 0.72]	0.22	0.50 [0.43, 0.56]	0.08
ACS-Inc	mlp	Perm.	50	1520	0.96 [0.93, 0.99]	0.90	0.76 [0.70, 0.82]	0.38	0.61 [0.55, 0.67]	0.16
ACS-Inc	mlp	Perm.	100	3039	0.98 [0.95, 0.99]	0.96	0.85 [0.80, 0.90]	0.60	0.73 [0.67, 0.78]	0.24
ACS-Inc	mlp	Perm.	200	6838	0.99 [0.98, 1.00]	0.98	0.88 [0.84, 0.93]	0.66	0.81 [0.76, 0.86]	0.48
ACS-Inc	mlp	Perm.	500	17474	1.00 [0.99, 1.00]	0.98	0.94 [0.90, 0.97]	0.82	0.90 [0.86, 0.93]	0.64
ACS-Inc	mlp	Perm.	1000	35709	0.99 [0.98, 1.00]	0.98	0.95 [0.91, 0.98]	0.84	0.93 [0.90, 0.97]	0.80
ACS-Inc	rf	Kernel	25	1251	0.85 [0.79, 0.90]	0.60	0.62 [0.55, 0.69]	0.16	0.48 [0.41, 0.55]	0.06
ACS-Inc	rf	Kernel	50	2501	0.93 [0.89, 0.97]	0.82	0.81 [0.74, 0.86]	0.54	0.69 [0.63, 0.76]	0.36
ACS-Inc	rf	Kernel	100	5001	0.96 [0.92, 0.98]	0.86	0.89 [0.84, 0.94]	0.70	0.82 [0.76, 0.88]	0.56
ACS-Inc	rf	Kernel	200	10001	0.97 [0.94, 1.00]	0.94	0.93 [0.88, 0.96]	0.76	0.87 [0.82, 0.92]	0.64
ACS-Inc	rf	Kernel	500	25001	0.98 [0.95, 1.00]	0.96	0.97 [0.94, 0.99]	0.90	0.94 [0.90, 0.98]	0.84
ACS-Inc	rf	Kernel	1000	50001	1.00 [0.99, 1.00]	0.98	0.99 [0.98, 1.00]	0.98	0.97 [0.93, 0.99]	0.92
ACS-Inc	rf	Perm.	25	760	0.90 [0.86, 0.94]	0.72	0.75 [0.68, 0.82]	0.40	0.70 [0.64, 0.76]	0.30
ACS-Inc	rf	Perm.	50	1520	0.94 [0.90, 0.97]	0.82	0.81 [0.74, 0.87]	0.54	0.78 [0.73, 0.84]	0.44
ACS-Inc	rf	Perm.	100	3039	0.95 [0.91, 0.98]	0.88	0.87 [0.81, 0.92]	0.64	0.84 [0.79, 0.89]	0.56
ACS-Inc	rf	Perm.	200	6838	0.97 [0.94, 0.99]	0.94	0.92 [0.87, 0.96]	0.74	0.89 [0.84, 0.93]	0.70
ACS-Inc	rf	Perm.	500	17474	0.99 [0.97, 1.00]	0.96	0.94 [0.90, 0.97]	0.84	0.92 [0.87, 0.96]	0.80
ACS-Inc	rf	Perm.	1000	35709	0.99 [0.97, 1.00]	0.96	0.95 [0.92, 0.98]	0.82	0.94 [0.90, 0.98]	0.84
ACS-Emp	gbm	Kernel	40	2001	0.98 [0.95, 1.00]	0.94	0.65 [0.58, 0.73]	0.20	0.32 [0.26, 0.38]	0.00
ACS-Emp	gbm	Kernel	64	3201	0.99 [0.96, 1.00]	0.98	0.81 [0.74, 0.87]	0.60	0.53 [0.46, 0.61]	0.12
ACS-Emp	gbm	Kernel	128	6401	0.99 [0.98, 1.00]	0.98	0.92 [0.88, 0.96]	0.74	0.71 [0.62, 0.78]	0.40
ACS-Emp	gbm	Kernel	256	12801	1.00 [0.99, 1.00]	0.98	0.94 [0.91, 0.97]	0.82	0.82 [0.75, 0.88]	0.54
ACS-Emp	gbm	Kernel	512	25601	1.00 [1.00, 1.00]	1.00	0.97 [0.94, 0.99]	0.90	0.88 [0.83, 0.93]	0.66
ACS-Emp	gbm	Kernel	1024	51201	1.00 [1.00, 1.00]	1.00	0.98 [0.96, 1.00]	0.94	0.94 [0.90, 0.96]	0.80
ACS-Emp	gbm	Kernel	2080	104001	1.00 [1.00, 1.00]	1.00	0.99 [0.98, 1.00]	0.98	0.97 [0.95, 0.99]	0.94
ACS-Emp	gbm	Kernel	5000	250001	1.00 [1.00, 1.00]	1.00	0.99 [0.98, 1.00]	0.98	0.99 [0.97, 1.00]	0.94
ACS-Emp	gbm	Perm.	40	787	0.98 [0.96, 1.00]	0.98	0.78 [0.71, 0.85]	0.56	0.64 [0.56, 0.70]	0.18
ACS-Emp	gbm	Perm.	64	787	0.98 [0.96, 1.00]	0.96	0.77 [0.70, 0.84]	0.50	0.62 [0.54, 0.69]	0.20
ACS-Emp	gbm	Perm.	128	2361	0.99 [0.97, 1.00]	0.98	0.87 [0.81, 0.92]	0.68	0.78 [0.71, 0.84]	0.44
ACS-Emp	gbm	Perm.	256	5508	0.99 [0.97, 1.00]	0.98	0.92 [0.88, 0.96]	0.74	0.84 [0.79, 0.90]	0.56
ACS-Emp	gbm	Perm.	512	11803	1.00 [0.99, 1.00]	0.98	0.93 [0.89, 0.97]	0.80	0.89 [0.84, 0.93]	0.66
ACS-Emp	gbm	Perm.	1024	24392	1.00 [0.99, 1.00]	1.00	0.97 [0.94, 0.99]	0.92	0.94 [0.91, 0.97]	0.80
ACS-Emp	gbm	Perm.	2080	49571	1.00 [1.00, 1.00]	1.00	0.98 [0.96, 0.99]	0.94	0.97 [0.93, 0.99]	0.92
ACS-Emp	gbm	Perm.	5000	118813	1.00 [1.00, 1.00]	1.00	0.99 [0.97, 1.00]	0.98	0.97 [0.94, 1.00]	0.92
ACS-Emp	lr	Kernel	40	2001	0.86 [0.80, 0.91]	0.66	0.70 [0.65, 0.74]	0.10	0.48 [0.41, 0.54]	0.02
ACS-Emp	lr	Kernel	64	3201	0.89 [0.84, 0.94]	0.74	0.88 [0.83, 0.92]	0.68	0.69 [0.61, 0.76]	0.38
ACS-Emp	lr	Kernel	128	6401	0.92 [0.87, 0.96]	0.82	0.94 [0.90, 0.97]	0.84	0.81 [0.74, 0.88]	0.56
ACS-Emp	lr	Kernel	256	12801	0.94 [0.90, 0.97]	0.86	0.96 [0.93, 0.99]	0.88	0.88 [0.83, 0.93]	0.64
ACS-Emp	lr	Kernel	512	25601	0.96 [0.92, 0.98]	0.88	0.98 [0.96, 1.00]	0.94	0.90 [0.85, 0.95]	0.76
ACS-Emp	lr	Kernel	1024	51201	0.96 [0.93, 0.99]	0.90	0.99 [0.97, 1.00]	0.96	0.93 [0.88, 0.97]	0.82
ACS-Emp	lr	Kernel	2080	104001	0.97 [0.94, 0.99]	0.94	0.99 [0.98, 1.00]	0.98	0.95 [0.92, 0.98]	0.88
ACS-Emp	lr	Kernel	5000	250001	0.98 [0.95, 1.00]	0.94	1.00 [1.00, 1.00]	1.00	0.96 [0.92, 0.99]	0.88
ACS-Emp	lr	Perm.	40	787	0.89 [0.84, 0.94]	0.74	0.80 [0.74, 0.86]	0.54	0.70 [0.63, 0.76]	0.22
ACS-Emp	lr	Perm.	64	787	0.89 [0.83, 0.94]	0.74	0.82 [0.77, 0.87]	0.50	0.72 [0.65, 0.78]	0.34
ACS-Emp	lr	Perm.	128	2361	0.92 [0.87, 0.96]	0.78	0.91 [0.87, 0.95]	0.74	0.80 [0.73, 0.87]	0.54
ACS-Emp	lr	Perm.	256	5508	0.93 [0.89, 0.97]	0.84	0.94 [0.91, 0.97]	0.80	0.89 [0.84, 0.94]	0.72
ACS-Emp	lr	Perm.	512	11803	0.95 [0.91, 0.98]	0.88	0.96 [0.92, 0.98]	0.90	0.91 [0.86, 0.95]	0.80
ACS-Emp	lr	Perm.	1024	24392	0.95 [0.91, 0.99]	0.90	0.97 [0.94, 0.99]	0.92	0.93 [0.89, 0.97]	0.84
ACS-Emp	lr	Perm.	2080	49571	0.96 [0.92, 0.99]	0.88	0.98 [0.97, 1.00]	0.94	0.95 [0.92, 0.98]	0.86
ACS-Emp	lr	Perm.	5000	118813	0.97 [0.94, 0.99]	0.90	0.99 [0.98, 1.00]	0.96	0.95 [0.91, 0.98]	0.86
ACS-Emp	mlp	Kernel	40	2001	0.86 [0.79, 0.91]	0.60	0.43 [0.36, 0.49]	0.02	0.27 [0.21, 0.35]	0.02
ACS-Emp	mlp	Kernel	64	3201	0.94 [0.90, 0.97]	0.84	0.63 [0.55, 0.69]	0.24	0.47 [0.39, 0.56]	0.14
ACS-Emp	mlp	Kernel	128	6401	0.95 [0.91, 0.99]	0.88	0.79 [0.72, 0.85]	0.48	0.65 [0.57, 0.74]	0.30
ACS-Emp	mlp	Kernel	256	12801	0.96 [0.93, 0.99]	0.92	0.87 [0.82, 0.92]	0.70	0.78 [0.71, 0.84]	0.44
ACS-Emp	mlp	Kernel	512	25601	0.98 [0.94, 1.00]	0.96	0.91 [0.85, 0.95]	0.76	0.85 [0.79, 0.91]	0.62
ACS-Emp	mlp	Kernel	1024	51201	0.98 [0.96, 1.00]	0.96	0.93 [0.88, 0.97]	0.84	0.91 [0.86, 0.95]	0.80
ACS-Emp	mlp	Kernel	2080	104001	0.99 [0.97, 1.00]	0.96	0.96 [0.92, 0.99]	0.92	0.93 [0.88, 0.98]	0.88
ACS-Emp	mlp	Kernel	5000	250001	0.99 [0.97, 1.00]	0.98	0.97 [0.94, 1.00]	0.92	0.95 [0.91, 0.99]	0.90
ACS-Emp	mlp	Perm.	40	787	0.90 [0.85, 0.95]	0.76	0.55 [0.48, 0.62]	0.14	0.45 [0.36, 0.53]	0.12
ACS-Emp	mlp	Perm.	64	787	0.91 [0.86, 0.95]	0.80	0.56 [0.49, 0.64]	0.16	0.46 [0.38, 0.55]	0.12
ACS-Emp	mlp	Perm.	128	2361	0.94 [0.89, 0.98]	0.86	0.70 [0.63, 0.77]	0.32	0.64 [0.56, 0.72]	0.26
ACS-Emp	mlp	Perm.	256	5508	0.96 [0.92, 0.99]	0.90	0.80 [0.75, 0.86]	0.44	0.73 [0.66, 0.81]	0.40
ACS-Emp	mlp	Perm.	512	11803	0.97 [0.93, 0.99]	0.92	0.85 [0.80, 0.91]	0.64	0.82 [0.76, 0.88]	0.54
ACS-Emp	mlp	Perm.	1024	24392	0.98 [0.95, 1.00]	0.92	0.91 [0.86, 0.95]	0.76	0.87 [0.81, 0.92]	0.68
ACS-Emp	mlp	Perm.	2080	49571	0.98 [0.94, 1.00]	0.96	0.93 [0.89, 0.97]	0.84	0.91 [0.87, 0.95]	0.72
ACS-Emp	mlp	Perm.	5000	118813	0.98 [0.96, 1.00]	0.96	0.95 [0.92, 0.99]	0.90	0.95 [0.91, 0.98]	0.90
ACS-Emp	rf	Kernel	40	2001	0.90 [0.84, 0.94]	0.78	0.58 [0.52, 0.65]	0.12	0.32 [0.27, 0.37]	0.00

Data	Model	Expl.	B	Rows	top-1 $\bar{p}$ [95% CI]	top-1 $\geq.9$	top-3 $\bar{p}$ [95% CI]	top-3 $\geq.9$	top-5 $\bar{p}$ [95% CI]	top-5 $\geq.9$
ACS-Emp	rf	Kernel	64	3201	0.92 [0.88, 0.96]	0.80	0.74 [0.68, 0.80]	0.38	0.54 [0.47, 0.62]	0.04
ACS-Emp	rf	Kernel	128	6401	0.95 [0.92, 0.98]	0.88	0.85 [0.80, 0.90]	0.64	0.74 [0.67, 0.80]	0.42
ACS-Emp	rf	Kernel	256	12801	0.96 [0.92, 0.99]	0.88	0.90 [0.85, 0.95]	0.72	0.83 [0.76, 0.88]	0.58
ACS-Emp	rf	Kernel	512	25601	0.97 [0.94, 0.99]	0.90	0.94 [0.91, 0.97]	0.82	0.88 [0.82, 0.92]	0.68
ACS-Emp	rf	Kernel	1024	51201	0.97 [0.94, 1.00]	0.92	0.96 [0.91, 0.99]	0.88	0.90 [0.85, 0.95]	0.72
ACS-Emp	rf	Kernel	2080	104001	0.99 [0.97, 1.00]	0.98	0.97 [0.95, 0.99]	0.94	0.93 [0.89, 0.97]	0.82
ACS-Emp	rf	Kernel	5000	250001	0.99 [0.97, 1.00]	0.98	0.98 [0.96, 1.00]	0.94	0.96 [0.93, 0.99]	0.90
ACS-Emp	rf	Perm.	40	787	0.91 [0.85, 0.96]	0.80	0.70 [0.63, 0.76]	0.32	0.54 [0.48, 0.61]	0.12
ACS-Emp	rf	Perm.	64	787	0.91 [0.86, 0.96]	0.76	0.70 [0.64, 0.76]	0.32	0.55 [0.49, 0.62]	0.12
ACS-Emp	rf	Perm.	128	2361	0.93 [0.88, 0.97]	0.86	0.82 [0.77, 0.87]	0.44	0.70 [0.63, 0.76]	0.30
ACS-Emp	rf	Perm.	256	5508	0.95 [0.91, 0.98]	0.86	0.88 [0.83, 0.93]	0.68	0.77 [0.70, 0.84]	0.44
ACS-Emp	rf	Perm.	512	11803	0.95 [0.91, 0.98]	0.88	0.91 [0.86, 0.95]	0.74	0.85 [0.79, 0.90]	0.60
ACS-Emp	rf	Perm.	1024	24392	0.97 [0.94, 0.99]	0.90	0.94 [0.90, 0.98]	0.86	0.89 [0.83, 0.93]	0.72
ACS-Emp	rf	Perm.	2080	49571	0.97 [0.95, 1.00]	0.92	0.96 [0.93, 0.99]	0.90	0.92 [0.88, 0.96]	0.74
ACS-Emp	rf	Perm.	5000	118813	0.98 [0.96, 0.99]	0.92	0.97 [0.94, 0.99]	0.90	0.94 [0.90, 0.97]	0.80
German	gbm	Kernel	60	3001	0.89 [0.84, 0.93]	0.70	0.66 [0.59, 0.73]	0.20	0.37 [0.31, 0.44]	0.02
German	gbm	Kernel	100	5001	0.93 [0.89, 0.96]	0.74	0.80 [0.75, 0.85]	0.44	0.61 [0.54, 0.68]	0.22
German	gbm	Kernel	200	10001	0.97 [0.95, 0.99]	0.90	0.90 [0.85, 0.94]	0.72	0.75 [0.68, 0.81]	0.42
German	gbm	Kernel	500	25001	0.99 [0.97, 1.00]	0.98	0.96 [0.93, 0.98]	0.88	0.88 [0.83, 0.93]	0.64
German	gbm	Kernel	1000	50001	0.99 [0.98, 1.00]	0.98	0.98 [0.96, 1.00]	0.92	0.92 [0.87, 0.96]	0.82
German	gbm	Kernel	2088	104401	1.00 [0.99, 1.00]	0.98	0.99 [0.96, 1.00]	0.98	0.95 [0.90, 0.98]	0.88
German	gbm	Kernel	5000	250001	1.00 [0.99, 1.00]	1.00	0.99 [0.98, 1.00]	0.98	0.97 [0.95, 0.99]	0.94
German	gbm	Perm.	60	1232	0.86 [0.81, 0.91]	0.58	0.65 [0.58, 0.73]	0.28	0.50 [0.43, 0.56]	0.16
German	gbm	Perm.	100	2465	0.89 [0.84, 0.93]	0.66	0.78 [0.72, 0.84]	0.42	0.61 [0.54, 0.68]	0.20
German	gbm	Perm.	200	4929	0.93 [0.89, 0.96]	0.80	0.85 [0.80, 0.89]	0.50	0.72 [0.65, 0.78]	0.32
German	gbm	Perm.	500	14788	0.95 [0.92, 0.98]	0.82	0.91 [0.86, 0.95]	0.72	0.85 [0.79, 0.90]	0.64
German	gbm	Perm.	1000	29576	0.97 [0.95, 0.99]	0.92	0.94 [0.91, 0.97]	0.80	0.91 [0.86, 0.94]	0.68
German	gbm	Perm.	2088	61616	0.98 [0.97, 1.00]	0.94	0.97 [0.94, 0.99]	0.94	0.95 [0.92, 0.98]	0.88
German	gbm	Perm.	5000	149111	0.99 [0.98, 1.00]	0.98	0.98 [0.96, 1.00]	0.98	0.97 [0.94, 0.99]	0.88
German	lr	Kernel	60	3001	0.94 [0.91, 0.96]	0.80	0.79 [0.72, 0.84]	0.50	0.65 [0.60, 0.71]	0.20
German	lr	Kernel	100	5001	0.97 [0.95, 0.99]	0.92	0.86 [0.81, 0.91]	0.64	0.79 [0.74, 0.84]	0.38
German	lr	Kernel	200	10001	1.00 [0.99, 1.00]	1.00	0.92 [0.88, 0.95]	0.70	0.87 [0.82, 0.92]	0.66
German	lr	Kernel	500	25001	1.00 [1.00, 1.00]	1.00	0.96 [0.94, 0.98]	0.86	0.94 [0.90, 0.97]	0.82
German	lr	Kernel	1000	50001	1.00 [1.00, 1.00]	1.00	0.97 [0.94, 0.99]	0.88	0.96 [0.92, 0.98]	0.90
German	lr	Kernel	2088	104401	1.00 [1.00, 1.00]	1.00	0.99 [0.97, 1.00]	0.96	0.97 [0.94, 0.99]	0.92
German	lr	Kernel	5000	250001	1.00 [1.00, 1.00]	1.00	1.00 [0.99, 1.00]	0.98	0.99 [0.98, 1.00]	0.94
German	lr	Perm.	60	1232	0.94 [0.91, 0.97]	0.80	0.84 [0.78, 0.88]	0.52	0.79 [0.73, 0.86]	0.46
German	lr	Perm.	100	2465	0.96 [0.94, 0.98]	0.86	0.87 [0.83, 0.92]	0.62	0.86 [0.80, 0.91]	0.62
German	lr	Perm.	200	4929	0.98 [0.97, 0.99]	0.96	0.90 [0.86, 0.94]	0.70	0.90 [0.85, 0.94]	0.70
German	lr	Perm.	500	14788	1.00 [0.99, 1.00]	0.98	0.94 [0.91, 0.97]	0.84	0.96 [0.92, 0.98]	0.88
German	lr	Perm.	1000	29576	1.00 [1.00, 1.00]	1.00	0.97 [0.95, 0.99]	0.88	0.96 [0.94, 0.99]	0.90
German	lr	Perm.	2088	61616	1.00 [1.00, 1.00]	1.00	0.98 [0.97, 0.99]	0.96	0.98 [0.95, 1.00]	0.92
German	lr	Perm.	5000	149111	1.00 [1.00, 1.00]	1.00	0.99 [0.99, 1.00]	1.00	0.99 [0.97, 1.00]	0.96
German	mlp	Kernel	60	3001	0.93 [0.90, 0.96]	0.74	0.70 [0.64, 0.76]	0.26	0.55 [0.48, 0.61]	0.18
German	mlp	Kernel	100	5001	0.98 [0.95, 0.99]	0.96	0.80 [0.74, 0.86]	0.50	0.72 [0.66, 0.78]	0.36
German	mlp	Kernel	200	10001	0.99 [0.98, 1.00]	0.98	0.90 [0.85, 0.94]	0.72	0.81 [0.75, 0.87]	0.50
German	mlp	Kernel	500	25001	0.99 [0.98, 1.00]	0.98	0.94 [0.89, 0.97]	0.84	0.90 [0.85, 0.94]	0.70
German	mlp	Kernel	1000	50001	1.00 [0.99, 1.00]	0.98	0.96 [0.92, 0.98]	0.86	0.94 [0.89, 0.97]	0.82
German	mlp	Kernel	2088	104401	0.99 [0.98, 1.00]	0.98	0.98 [0.96, 0.99]	0.92	0.96 [0.92, 0.98]	0.88
German	mlp	Kernel	5000	250001	1.00 [0.99, 1.00]	1.00	0.99 [0.97, 1.00]	0.92	0.97 [0.94, 0.99]	0.92
German	mlp	Perm.	60	1232	0.92 [0.88, 0.95]	0.74	0.73 [0.65, 0.79]	0.34	0.64 [0.57, 0.71]	0.26
German	mlp	Perm.	100	2465	0.95 [0.92, 0.97]	0.84	0.80 [0.74, 0.86]	0.46	0.72 [0.65, 0.78]	0.34
German	mlp	Perm.	200	4929	0.97 [0.95, 0.99]	0.94	0.87 [0.81, 0.92]	0.68	0.80 [0.75, 0.86]	0.52
German	mlp	Perm.	500	14788	0.99 [0.98, 1.00]	0.98	0.90 [0.84, 0.95]	0.76	0.88 [0.83, 0.92]	0.64
German	mlp	Perm.	1000	29576	0.99 [0.97, 1.00]	0.98	0.92 [0.87, 0.97]	0.78	0.90 [0.86, 0.94]	0.66
German	mlp	Perm.	2088	61616	0.99 [0.98, 1.00]	0.98	0.95 [0.91, 0.98]	0.88	0.94 [0.90, 0.97]	0.86
German	mlp	Perm.	5000	149111	0.99 [0.98, 1.00]	0.98	0.96 [0.93, 0.99]	0.88	0.95 [0.91, 0.98]	0.88
German	rf	Kernel	60	3001	0.96 [0.93, 0.99]	0.92	0.71 [0.65, 0.77]	0.30	0.50 [0.43, 0.57]	0.06
German	rf	Kernel	100	5001	0.98 [0.95, 1.00]	0.96	0.82 [0.75, 0.88]	0.54	0.68 [0.60, 0.75]	0.34
German	rf	Kernel	200	10001	0.99 [0.97, 1.00]	0.96	0.89 [0.83, 0.95]	0.74	0.78 [0.72, 0.85]	0.50
German	rf	Kernel	500	25001	0.99 [0.97, 1.00]	0.96	0.93 [0.88, 0.97]	0.86	0.86 [0.81, 0.91]	0.64
German	rf	Kernel	1000	50001	1.00 [0.99, 1.00]	0.98	0.93 [0.88, 0.97]	0.84	0.91 [0.86, 0.95]	0.76
German	rf	Kernel	2088	104401	1.00 [0.99, 1.00]	1.00	0.95 [0.90, 0.98]	0.88	0.95 [0.91, 0.98]	0.82
German	rf	Kernel	5000	250001	1.00 [1.00, 1.00]	1.00	0.97 [0.93, 0.99]	0.90	0.97 [0.95, 0.99]	0.92
German	rf	Perm.	60	1232	0.96 [0.93, 0.99]	0.92	0.73 [0.66, 0.79]	0.34	0.58 [0.51, 0.65]	0.20
German	rf	Perm.	100	2465	0.97 [0.94, 1.00]	0.92	0.81 [0.74, 0.86]	0.54	0.70 [0.63, 0.77]	0.32
German	rf	Perm.	200	4929	0.98 [0.94, 1.00]	0.94	0.86 [0.81, 0.91]	0.66	0.78 [0.71, 0.84]	0.50
German	rf	Perm.	500	14788	0.99 [0.96, 1.00]	0.96	0.91 [0.86, 0.96]	0.82	0.85 [0.78, 0.91]	0.64
German	rf	Perm.	1000	29576	0.99 [0.97, 1.00]	0.96	0.93 [0.88, 0.97]	0.84	0.90 [0.85, 0.94]	0.72
German	rf	Perm.	2088	61616	0.99 [0.98, 1.00]	0.98	0.94 [0.89, 0.98]	0.88	0.93 [0.90, 0.97]	0.80
German	rf	Perm.	5000	149111	0.99 [0.98, 1.00]	0.98	0.95 [0.91, 0.99]	0.90	0.95 [0.92, 0.98]	0.82
GMSC	gbm	Kernel	25	1251	0.89 [0.84, 0.93]	0.70	0.74 [0.68, 0.79]	0.34	0.52 [0.47, 0.58]	0.06
GMSC	gbm	Kernel	50	2501	0.95 [0.91, 0.98]	0.84	0.90 [0.85, 0.95]	0.76	0.78 [0.72, 0.83]	0.30
GMSC	gbm	Kernel	100	5001	0.97 [0.95, 0.99]	0.88	0.94 [0.90, 0.97]	0.84	0.88 [0.84, 0.93]	0.64
GMSC	gbm	Kernel	200	10001	0.98 [0.96, 1.00]	0.96	0.97 [0.94, 0.99]	0.88	0.94 [0.90, 0.97]	0.84
GMSC	gbm	Kernel	500	25001	1.00 [0.99, 1.00]	0.98	0.99 [0.97, 1.00]	0.98	0.97 [0.95, 0.99]	0.88
GMSC	gbm	Kernel	1000	50001	1.00 [1.00, 1.00]	1.00	1.00 [0.99, 1.00]	1.00	0.99 [0.98, 1.00]	0.96
GMSC	gbm	Perm.	25	707	0.90 [0.86, 0.94]	0.66	0.83 [0.77, 0.88]	0.54	0.73 [0.67, 0.79]	0.34

Data	Model	Expl.	B	Rows	top-1 $\bar{p}$ [95% CI]	top-1 $\geq.9$	top-3 $\bar{p}$ [95% CI]	top-3 $\geq.9$	top-5 $\bar{p}$ [95% CI]	top-5 $\geq.9$
GMSC	gbm	Perm.	50	1414	0.92 [0.88, 0.96]	0.80	0.88 [0.82, 0.92]	0.70	0.81 [0.76, 0.86]	0.44
GMSC	gbm	Perm.	100	2828	0.94 [0.90, 0.98]	0.84	0.90 [0.86, 0.95]	0.76	0.89 [0.85, 0.93]	0.64
GMSC	gbm	Perm.	200	6363	0.97 [0.94, 0.99]	0.88	0.94 [0.90, 0.97]	0.84	0.92 [0.88, 0.95]	0.80
GMSC	gbm	Perm.	500	16262	0.98 [0.96, 1.00]	0.94	0.97 [0.94, 0.99]	0.92	0.96 [0.92, 0.98]	0.86
GMSC	gbm	Perm.	1000	33231	0.99 [0.98, 1.00]	0.96	0.98 [0.96, 1.00]	0.94	0.97 [0.94, 0.99]	0.92
GMSC	lr	Kernel	25	1251	0.96 [0.94, 0.98]	0.94	0.86 [0.82, 0.89]	0.54	0.71 [0.66, 0.77]	0.18
GMSC	lr	Kernel	50	2501	0.99 [0.98, 1.00]	0.96	0.96 [0.94, 0.98]	0.88	0.87 [0.82, 0.92]	0.72
GMSC	lr	Kernel	100	5001	0.99 [0.98, 1.00]	0.98	0.98 [0.95, 1.00]	0.96	0.93 [0.89, 0.96]	0.78
GMSC	lr	Kernel	200	10001	1.00 [1.00, 1.00]	1.00	0.99 [0.98, 1.00]	0.98	0.95 [0.90, 0.98]	0.88
GMSC	lr	Kernel	500	25001	1.00 [1.00, 1.00]	1.00	0.99 [0.98, 1.00]	0.98	0.98 [0.96, 1.00]	0.94
GMSC	lr	Kernel	1000	50001	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00
GMSC	lr	Perm.	25	707	0.99 [0.97, 1.00]	0.96	0.99 [0.97, 1.00]	0.94	0.97 [0.94, 0.99]	0.92
GMSC	lr	Perm.	50	1414	0.99 [0.99, 1.00]	0.98	0.99 [0.97, 1.00]	0.96	0.98 [0.94, 1.00]	0.96
GMSC	lr	Perm.	100	2828	1.00 [1.00, 1.00]	1.00	0.99 [0.99, 1.00]	0.98	0.99 [0.97, 1.00]	0.96
GMSC	lr	Perm.	200	6363	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00	0.99 [0.98, 1.00]	0.96
GMSC	lr	Perm.	500	16262	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00	1.00 [0.99, 1.00]	0.98
GMSC	lr	Perm.	1000	33231	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00	1.00 [1.00, 1.00]	1.00
GMSC	mlp	Kernel	25	1251	0.93 [0.90, 0.96]	0.82	0.62 [0.56, 0.67]	0.12	0.57 [0.50, 0.63]	0.14
GMSC	mlp	Kernel	50	2501	0.98 [0.97, 1.00]	0.96	0.77 [0.72, 0.82]	0.34	0.75 [0.69, 0.82]	0.36
GMSC	mlp	Kernel	100	5001	0.99 [0.97, 1.00]	0.98	0.85 [0.80, 0.89]	0.52	0.86 [0.80, 0.90]	0.62
GMSC	mlp	Kernel	200	10001	1.00 [0.99, 1.00]	0.98	0.90 [0.85, 0.94]	0.72	0.92 [0.88, 0.95]	0.74
GMSC	mlp	Kernel	500	25001	1.00 [1.00, 1.00]	1.00	0.95 [0.92, 0.98]	0.86	0.96 [0.93, 0.99]	0.94
GMSC	mlp	Kernel	1000	50001	1.00 [1.00, 1.00]	1.00	0.98 [0.96, 1.00]	0.96	0.99 [0.99, 1.00]	0.96
GMSC	mlp	Perm.	25	707	0.95 [0.92, 0.98]	0.82	0.71 [0.65, 0.77]	0.28	0.65 [0.58, 0.72]	0.22
GMSC	mlp	Perm.	50	1414	0.97 [0.94, 0.99]	0.88	0.78 [0.72, 0.83]	0.38	0.76 [0.68, 0.82]	0.42
GMSC	mlp	Perm.	100	2828	0.98 [0.96, 1.00]	0.96	0.85 [0.80, 0.89]	0.56	0.84 [0.77, 0.89]	0.56
GMSC	mlp	Perm.	200	6363	0.99 [0.98, 1.00]	0.98	0.88 [0.83, 0.92]	0.64	0.89 [0.85, 0.93]	0.68
GMSC	mlp	Perm.	500	16262	1.00 [0.99, 1.00]	0.98	0.92 [0.87, 0.95]	0.72	0.94 [0.91, 0.97]	0.82
GMSC	mlp	Perm.	1000	33231	1.00 [0.99, 1.00]	0.98	0.95 [0.91, 0.98]	0.88	0.97 [0.94, 0.99]	0.90
GMSC	rf	Kernel	25	1251	0.83 [0.76, 0.88]	0.62	0.56 [0.50, 0.62]	0.00	0.34 [0.29, 0.39]	0.00
GMSC	rf	Kernel	50	2501	0.92 [0.87, 0.96]	0.80	0.79 [0.73, 0.85]	0.48	0.58 [0.52, 0.64]	0.10
GMSC	rf	Kernel	100	5001	0.95 [0.91, 0.98]	0.88	0.90 [0.86, 0.94]	0.64	0.71 [0.64, 0.77]	0.30
GMSC	rf	Kernel	200	10001	0.96 [0.93, 0.99]	0.90	0.95 [0.91, 0.97]	0.82	0.82 [0.77, 0.88]	0.58
GMSC	rf	Kernel	500	25001	0.97 [0.94, 0.99]	0.92	0.98 [0.96, 1.00]	0.96	0.90 [0.85, 0.94]	0.70
GMSC	rf	Kernel	1000	50001	0.99 [0.97, 1.00]	0.98	1.00 [1.00, 1.00]	1.00	0.98 [0.96, 0.99]	0.90
GMSC	rf	Perm.	25	707	0.85 [0.79, 0.91]	0.64	0.68 [0.62, 0.73]	0.18	0.47 [0.41, 0.54]	0.04
GMSC	rf	Perm.	50	1414	0.90 [0.85, 0.94]	0.70	0.77 [0.72, 0.83]	0.38	0.59 [0.52, 0.65]	0.14
GMSC	rf	Perm.	100	2828	0.92 [0.86, 0.96]	0.80	0.84 [0.79, 0.89]	0.52	0.68 [0.61, 0.74]	0.24
GMSC	rf	Perm.	200	6363	0.94 [0.89, 0.98]	0.86	0.91 [0.87, 0.95]	0.76	0.77 [0.71, 0.83]	0.38
GMSC	rf	Perm.	500	16262	0.96 [0.92, 0.99]	0.90	0.95 [0.92, 0.98]	0.86	0.84 [0.78, 0.90]	0.62
GMSC	rf	Perm.	1000	33231	0.96 [0.92, 0.99]	0.88	0.98 [0.95, 1.00]	0.96	0.88 [0.84, 0.93]	0.68
Pima	gbm	Kernel	20	1001	0.90 [0.86, 0.94]	0.74	0.62 [0.56, 0.69]	0.18	0.45 [0.40, 0.51]	0.02
Pima	gbm	Kernel	40	2001	0.96 [0.93, 0.99]	0.92	0.81 [0.76, 0.86]	0.46	0.68 [0.62, 0.75]	0.30
Pima	gbm	Kernel	80	4001	0.97 [0.94, 0.99]	0.94	0.89 [0.84, 0.93]	0.70	0.82 [0.76, 0.88]	0.58
Pima	gbm	Kernel	160	8001	0.98 [0.96, 1.00]	0.96	0.94 [0.90, 0.97]	0.80	0.91 [0.86, 0.95]	0.70
Pima	gbm	Kernel	240	12001	0.99 [0.96, 1.00]	0.96	0.96 [0.93, 0.99]	0.88	0.95 [0.92, 0.98]	0.84
Pima	gbm	Perm.	20	790	0.92 [0.88, 0.96]	0.78	0.72 [0.66, 0.79]	0.34	0.67 [0.60, 0.74]	0.30
Pima	gbm	Perm.	40	1580	0.94 [0.90, 0.98]	0.86	0.83 [0.78, 0.88]	0.48	0.76 [0.69, 0.82]	0.42
Pima	gbm	Perm.	80	3160	0.97 [0.94, 0.99]	0.90	0.85 [0.80, 0.91]	0.60	0.84 [0.78, 0.89]	0.56
Pima	gbm	Perm.	160	7109	0.97 [0.95, 1.00]	0.94	0.92 [0.88, 0.96]	0.78	0.88 [0.83, 0.93]	0.70
Pima	gbm	Perm.	240	11059	0.97 [0.94, 1.00]	0.94	0.94 [0.90, 0.97]	0.82	0.91 [0.87, 0.95]	0.78
Pima	lr	Kernel	20	1001	0.98 [0.96, 0.99]	0.94	0.87 [0.82, 0.91]	0.66	0.78 [0.72, 0.84]	0.44
Pima	lr	Kernel	40	2001	1.00 [1.00, 1.00]	1.00	0.95 [0.91, 0.98]	0.86	0.90 [0.86, 0.94]	0.66
Pima	lr	Kernel	80	4001	1.00 [1.00, 1.00]	1.00	0.96 [0.93, 0.99]	0.90	0.93 [0.89, 0.97]	0.80
Pima	lr	Kernel	160	8001	1.00 [1.00, 1.00]	1.00	0.98 [0.96, 1.00]	0.92	0.97 [0.94, 1.00]	0.94
Pima	lr	Kernel	240	12001	1.00 [1.00, 1.00]	1.00	0.99 [0.97, 1.00]	0.96	0.98 [0.95, 1.00]	0.96
Pima	lr	Perm.	20	790	1.00 [0.99, 1.00]	1.00	0.95 [0.91, 0.98]	0.84	0.92 [0.88, 0.96]	0.78
Pima	lr	Perm.	40	1580	1.00 [1.00, 1.00]	1.00	0.95 [0.92, 0.98]	0.86	0.94 [0.90, 0.97]	0.82
Pima	lr	Perm.	80	3160	1.00 [1.00, 1.00]	1.00	0.97 [0.94, 0.99]	0.90	0.96 [0.92, 0.98]	0.86
Pima	lr	Perm.	160	7109	1.00 [1.00, 1.00]	1.00	0.98 [0.95, 0.99]	0.90	0.98 [0.95, 0.99]	0.94
Pima	lr	Perm.	240	11059	1.00 [1.00, 1.00]	1.00	0.98 [0.96, 1.00]	0.92	0.98 [0.96, 1.00]	0.96
Pima	mlp	Kernel	20	1001	0.91 [0.88, 0.95]	0.76	0.60 [0.53, 0.67]	0.16	0.43 [0.37, 0.49]	0.02
Pima	mlp	Kernel	40	2001	0.98 [0.95, 0.99]	0.96	0.79 [0.72, 0.85]	0.50	0.65 [0.58, 0.71]	0.24
Pima	mlp	Kernel	80	4001	0.98 [0.96, 1.00]	0.96	0.89 [0.84, 0.94]	0.72	0.80 [0.74, 0.86]	0.52
Pima	mlp	Kernel	160	8001	1.00 [0.99, 1.00]	1.00	0.93 [0.89, 0.97]	0.84	0.90 [0.85, 0.94]	0.70
Pima	mlp	Kernel	240	12001	1.00 [1.00, 1.00]	1.00	0.95 [0.92, 0.98]	0.86	0.95 [0.92, 0.98]	0.88
Pima	mlp	Perm.	20	790	0.91 [0.86, 0.95]	0.78	0.69 [0.62, 0.76]	0.30	0.55 [0.49, 0.61]	0.08
Pima	mlp	Perm.	40	1580	0.95 [0.91, 0.98]	0.86	0.76 [0.69, 0.83]	0.48	0.65 [0.58, 0.70]	0.20
Pima	mlp	Perm.	80	3160	0.97 [0.94, 0.99]	0.92	0.82 [0.77, 0.88]	0.54	0.74 [0.68, 0.81]	0.36
Pima	mlp	Perm.	160	7109	0.98 [0.95, 1.00]	0.96	0.89 [0.84, 0.94]	0.70	0.81 [0.75, 0.87]	0.50
Pima	mlp	Perm.	240	11059	0.98 [0.95, 1.00]	0.96	0.89 [0.84, 0.94]	0.72	0.86 [0.80, 0.91]	0.64
Pima	rf	Kernel	20	1001	0.95 [0.92, 0.97]	0.84	0.75 [0.69, 0.81]	0.36	0.52 [0.47, 0.58]	0.04
Pima	rf	Kernel	40	2001	0.99 [0.97, 1.00]	0.96	0.89 [0.84, 0.93]	0.72	0.74 [0.68, 0.80]	0.36
Pima	rf	Kernel	80	4001	1.00 [0.99, 1.00]	0.98	0.95 [0.91, 0.98]	0.82	0.85 [0.79, 0.90]	0.60
Pima	rf	Kernel	160	8001	1.00 [1.00, 1.00]	1.00	0.97 [0.93, 0.99]	0.90	0.91 [0.86, 0.96]	0.80
Pima	rf	Kernel	240	12001	1.00 [1.00, 1.00]	1.00	0.98 [0.97, 1.00]	0.96	0.94 [0.90, 0.98]	0.84
Pima	rf	Perm.	20	790	0.97 [0.95, 0.99]	0.88	0.85 [0.80, 0.90]	0.58	0.68 [0.62, 0.74]	0.20
Pima	rf	Perm.	40	1580	0.98 [0.95, 0.99]	0.94	0.90 [0.84, 0.94]	0.72	0.78 [0.71, 0.83]	0.38

Data	Model	Expl.	B	Rows	top-1 $\bar{p}$ [95% CI]	top-1 $\geq.9$	top-3 $\bar{p}$ [95% CI]	top-3 $\geq.9$	top-5 $\bar{p}$ [95% CI]	top-5 $\geq.9$
Pima	rf	Perm.	80	3160	0.99 [0.99, 1.00]	1.00	0.92 [0.87, 0.96]	0.80	0.84 [0.78, 0.90]	0.58
Pima	rf	Perm.	160	7109	1.00 [0.99, 1.00]	1.00	0.95 [0.91, 0.98]	0.84	0.89 [0.83, 0.94]	0.74
Pima	rf	Perm.	240	11059	1.00 [1.00, 1.00]	1.00	0.95 [0.92, 0.98]	0.86	0.90 [0.85, 0.94]	0.74

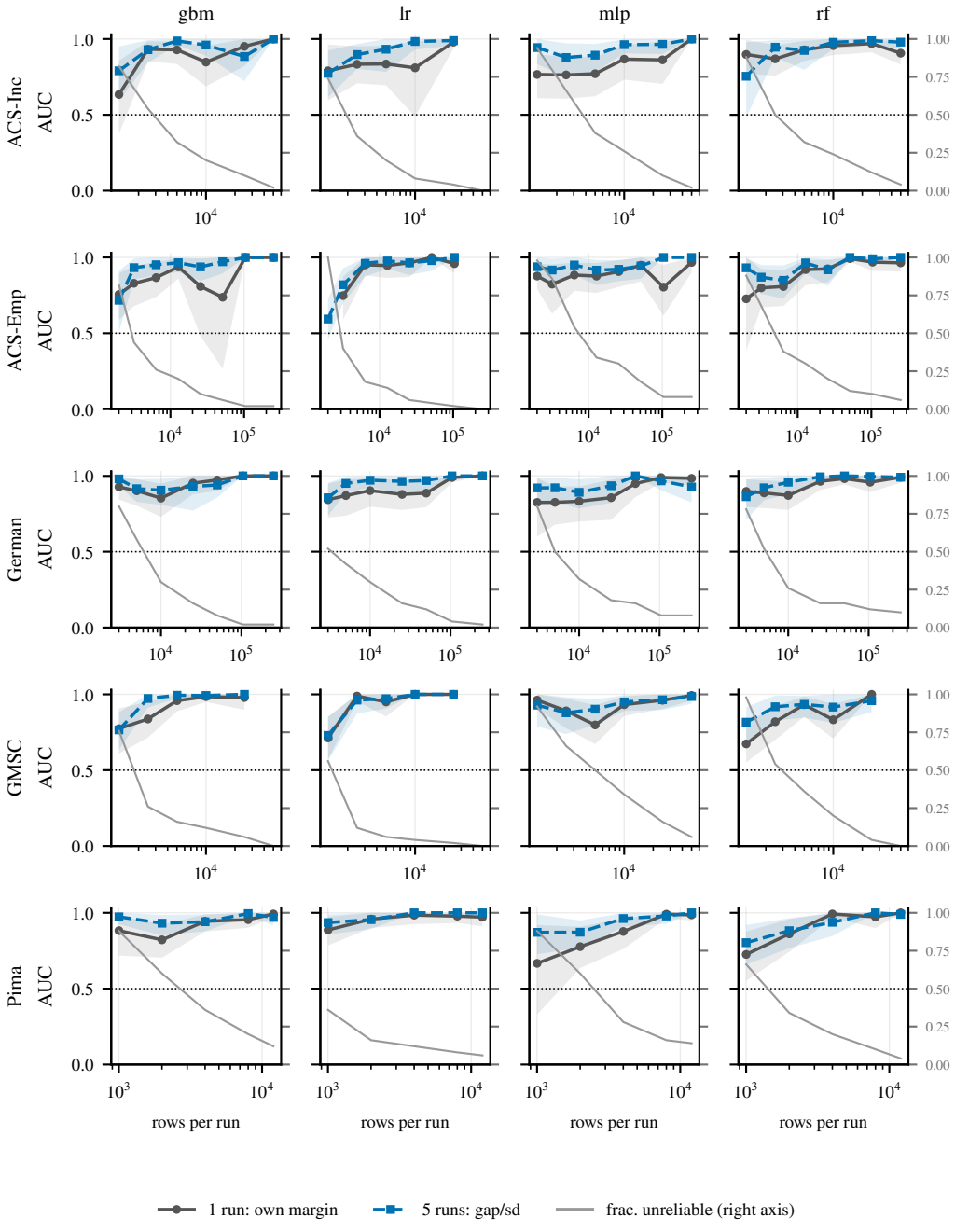


Figure 5: (Exploratory) AUC of per-instance diagnostics for flagging unreliable instances.

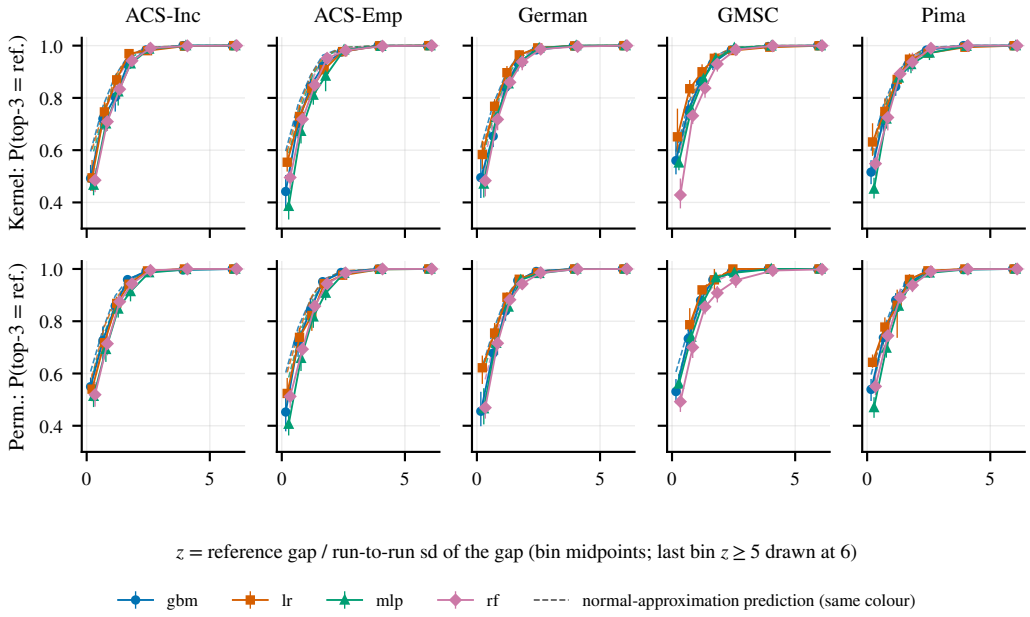


Figure 6: Top-3 match probability against z = reference gap/run-to-run sd of the gap, one line per model, 95% CI, for each dataset and explainer. Dashed: the normal-approximation reference shape $\Phi(z)$ for one competing pair (computed by the plotting script, not fitted). The curves lie close to it because z uses the sd of the same runs: the relation is largely built in and is not an independent test.

A.3 Match probability against z, per cell

B Default `l1_reg`, reference check and log-odds

C Pre-specified choices

The study plan is part of the delivery: `plan/study_plan_at_launch.md` is the version in force when the main grid was launched, `plan/study_plan.md` the final version, and `plan/CHANGES.md` lists every revision with its commit and time. The status of each analysis in this paper, read from those files and from the commit log of the analysis code, is in Table 10. "Planned subset" means the plan named the analysis but the paper's selection of cells or pairs was made later; "post-hoc" means the plan did not contain it. For the post-hoc items we cannot show whether they were written before or after the results were seen, so they should be read as exploratory.

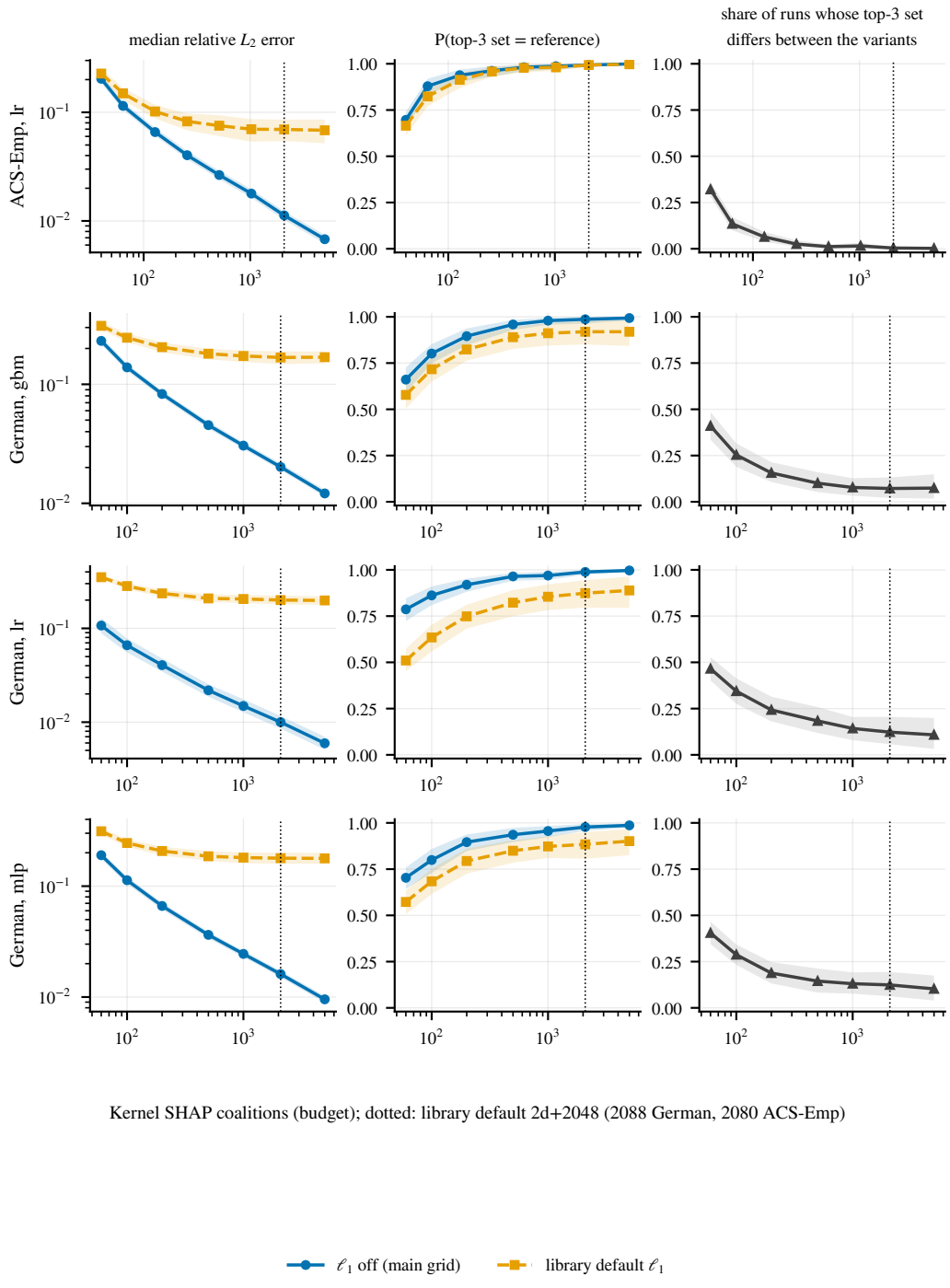


Figure 7: German Credit (three models) and ACS Employment logistic regression, Kernel SHAP with `l1_reg=False` (main grid) versus the library default `l1_reg`: error and top-3 match against the reference (20 000 coalitions for German Credit, exact for ACS Employment). The default variant plateaus because its target is the sparsified regression, a bias, not run-to-run noise.

Data	Model	Inst.	Pairs	P(same top-3 set), pair of retrained models 50% subsample	80% (main)	95%
ACS-Inc	gbm	50	500	0.61 [0.52, 0.70]	0.73 [0.65, 0.81]	0.79 [0.71, 0.87]
ACS-Inc	lr	50	500	0.88 [0.82, 0.93]	0.96 [0.92, 0.99]	0.97 [0.93, 1.00]
ACS-Inc	mlp	50	500	0.55 [0.46, 0.64]	0.63 [0.54, 0.71]	0.65 [0.56, 0.75]
ACS-Inc	rf	50	500	0.64 [0.56, 0.74]	0.72 [0.64, 0.80]	0.72 [0.62, 0.80]
ACS-Emp	gbm	50	500	–	0.69 [0.59, 0.78]	–
ACS-Emp	lr	50	500	0.89 [0.82, 0.94]	0.91 [0.86, 0.96]	0.98 [0.95, 1.00]
ACS-Emp	mlp	50	500	0.44 [0.35, 0.53]	0.45 [0.36, 0.54]	0.49 [0.40, 0.58]
ACS-Emp	rf	50	500	0.48 [0.40, 0.56]	0.57 [0.49, 0.66]	0.66 [0.58, 0.73]
German	gbm	50	500	0.11 [0.08, 0.15]	0.27 [0.19, 0.35]	0.51 [0.42, 0.61]
German	lr	50	500	0.11 [0.08, 0.14]	0.34 [0.28, 0.41]	0.68 [0.60, 0.76]
German	mlp	50	500	0.05 [0.03, 0.07]	0.09 [0.06, 0.11]	0.18 [0.14, 0.24]
German	rf	50	500	0.12 [0.07, 0.20]	0.30 [0.23, 0.38]	0.33 [0.26, 0.41]
GMSC	gbm	50	500	–	0.62 [0.52, 0.72]	–
GMSC	lr	50	500	–	0.89 [0.83, 0.94]	–
GMSC	mlp	50	500	–	0.19 [0.15, 0.23]	–
GMSC	rf	50	500	–	0.49 [0.41, 0.56]	–
Pima	gbm	50	500	–	0.33 [0.27, 0.42]	–
Pima	lr	50	500	–	0.61 [0.53, 0.70]	–
Pima	mlp	50	500	–	0.34 [0.27, 0.42]	–
Pima	rf	50	500	–	0.39 [0.32, 0.46]	–

Table 6: Retrain-subsample sensitivity: probability that two retrained models give the same top-3 set, for 50%, 80% (main protocol) and 95% training subsamples (five retrains, 10 pairs per instance, 95% CI over instances; the pairs are not independent). "–": not run (the 50% and 95% fractions were run for German Credit and ACS Income, and for ACS Employment only for logistic regression, forest and MLP). Exact explanations for $d \leq 16$, one Kernel run of 10 000 coalitions for German Credit.

Data	Model	P(same top-3 set) two 80% retrained models	same retrained model, two Kernel seeds
German	gbm	0.27 [0.19, 0.35]	0.98 [0.96, 0.99]
German	lr	0.34 [0.28, 0.41]	0.99 [0.98, 1.00]
German	mlp	0.09 [0.06, 0.11]	0.98 [0.96, 0.99]
German	rf	0.30 [0.23, 0.38]	0.98 [0.96, 1.00]

Table 7: German Credit: the estimator's contribution to retrain disagreement. Two 80%-retrained models explained with one Kernel run each, against the same retrained model explained with two Kernel seeds.

Data	Model	Inst.	median ref. vs enum.	max ref. vs enum.	median ref. rel. SE	top-3 equal	top-5 equal
German	gbm	13	0.0016	0.0024	0.0016	13/13	13/13
German	lr	13	0.0007	0.0021	0.0008	13/13	13/13

Table 8: German Credit reference check, per model: relative ℓ_2 distance of the 20 000-coalition reference (mean of 10 Kernel runs) to the exact vector by enumeration of 2^{20} coalitions (median and maximum over the instances checked), the reference's median relative standard error, and the number of instances whose top-3 and top-5 sets coincide.

Data	Model	Same top-3 of ϕ^* prob. vs log-odds	Explainer	P(top-3 = ref.), smallest budget prob.	log-odds	P(top-3 = ref.), largest budget prob.	log-odds
ACS-Emp	gbm	0.78 [0.66, 0.88]	Kernel	0.65	0.72	0.99	0.99
			Perm.	0.78	0.82	0.99	0.98
German	gbm	0.50 [0.36, 0.64]	Kernel	0.66	0.73	0.99	0.98
			Perm.	0.65	0.76	0.98	0.96
German	lr	0.86 [0.76, 0.94]	Kernel	0.79	1.00	1.00	1.00
			Perm.	0.84	1.00	0.99	1.00
German	mlp	0.82 [0.70, 0.92]	Kernel	0.70	0.73	0.99	0.97
			Perm.	0.73	0.74	0.96	0.95
Pima	gbm	0.50 [0.36, 0.64]	Kernel	0.62	0.78	0.96	1.00
			Perm.	0.72	0.91	0.94	0.98

Table 9: Explained output: probability of the positive class versus log-odds. Fraction of instances whose exact (reference) top-3 sets coincide, with 95% CI, and the top-3 match probability to each output's own reference at the smallest and largest budget. For German logistic regression the log-odds game is linear, so Kernel SHAP is exact there. Only these five cells were run (ACS Employment gbm; German gbm, lr, mlp; Pima gbm).

Analysis	Status
Measures (relative ℓ_2, top-k match and Jaccard, Kendall τ, RBO, Nogueira, sign flips, margin); $k \in \{1, 3, 5\}$, $k = 3$ for headlines	pre-specified
Population-level adequacy criterion, instance-cluster bootstrap, error-versus-budget slope fits	pre-specified
Headline cells: German Credit and ACS Employment at the library-default budget	pre-specified; ACS Employment added before any result
Comparison standards (retraining, background redraws, null)	pre-specified; run on a subset of cells
Default `l1_reg` and log-odds output	pre-specified as sensitivity checks on a subset of cells; which cells chosen later
Per-instance diagnostic	pre-specified as optional, exploratory
Repeat versus enlarge	pre-specified as a comparison; pair selection (budget ratio within $[0.9, 1.1]$) set after the budgets were fixed
50 instances (plan: 100), ACS Employment in the core, five retrain seeds, 50-tree forest, budgets not in ratio 4	design changed at calibration, recorded after launch and before any result
Median and blow-up summaries, slope fits restricted to the Monte-Carlo-like range, skipping local slopes between budgets with equal rows, the gap-over-run-noise form of the margin analysis	post-hoc
Explainer disagreement, Covert–Lee variance comparison, Arrhythmia	planned, not run

Table 10: Status of the analyses relative to the study plan.